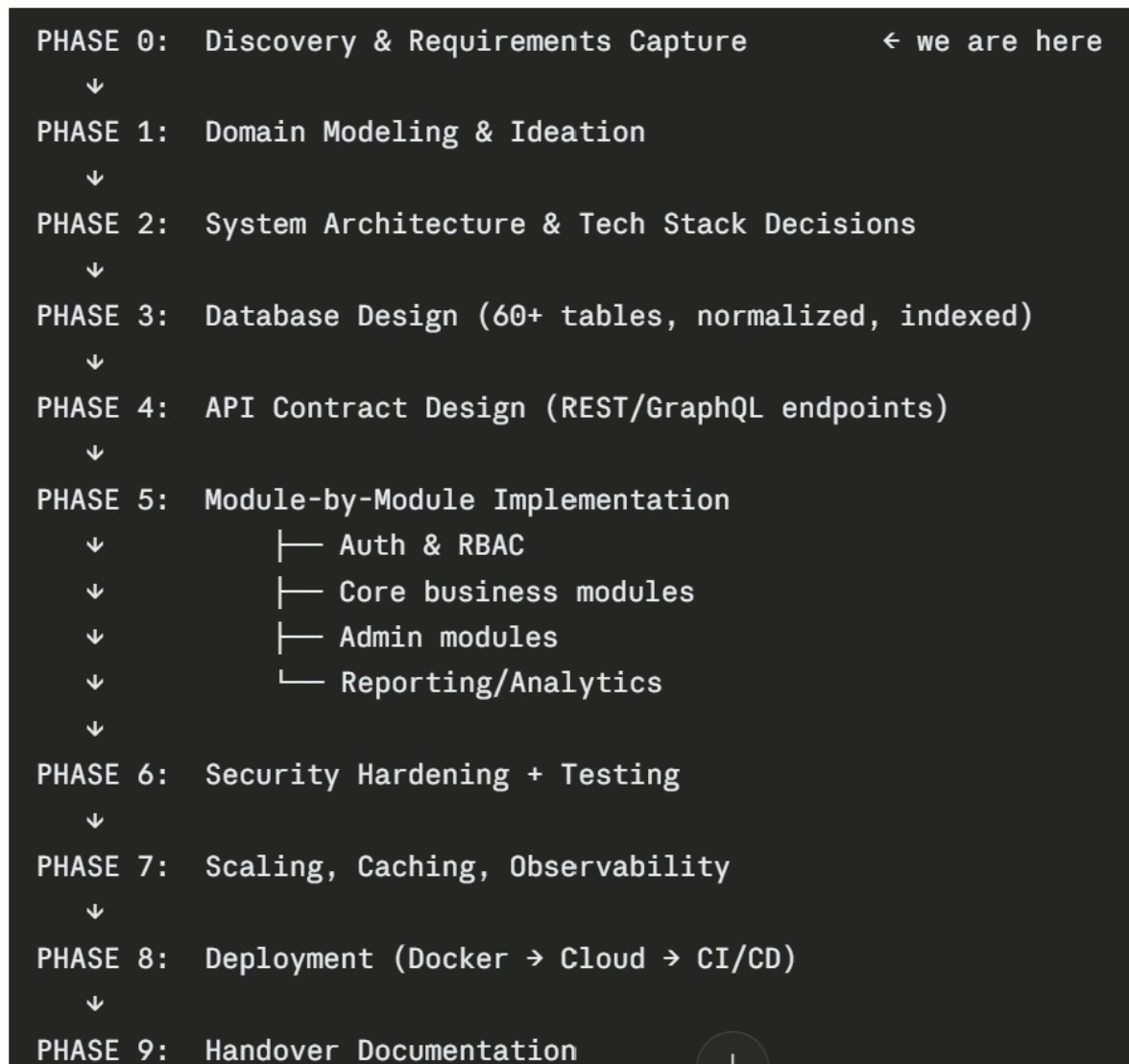


Today we are going to create history, today we are going to create a new project from basic to advance code as starting from scratch to advance admin modules.



As we are forwarding before i give very lengthy PROJECT DESCRIPTION so that you think complex things for this project and for another requirements. please follow all the things carefully.

Because this not only the basic prototype or college project or not like as major final semester project but this is somethings to be special for you and also it is very important for me as well for my future. Currently i am doing my internship at organization WITH THE DESIGNATION OF SOFTWARE DEVELOPER and this is my

organization project. so take it very serious and highest complex task priority.

HIGHEST Priority -----> "This is INDUSTRY GRADE ACTIVE USER BASED PROJECT THAT WILL BE USED BY across multiple roles of peoples AND WE HAVE GETTING CHANCE TO DEVELOP IT IN MY ORGANIZATION."

we have to develop it from Definition, Ideas, IDEATION, DESIGNS, database design with over the 60+ tables SCHEMAS IN SQL {highest and very much IMPORTANT} , all the require modules development and codes, scale and Grow and at last till DEPLOYEMNT.

Are you ready for this?

Can i give you my entire project Description?

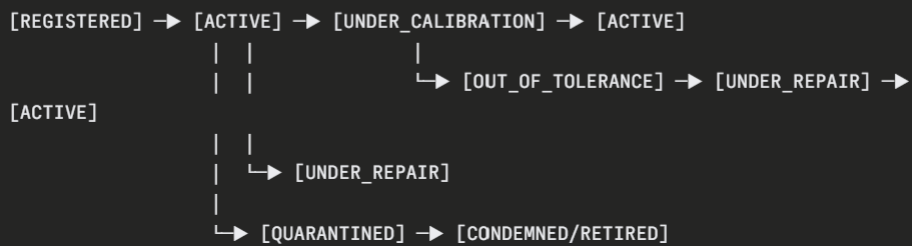
📖 Section 5 — Critical State Machines

These define how things move through your system. Without explicit state machines, your team will write inconsistent transitions and create data corruption bugs.

5.1 Job Request Lifecycle



5.2 Equipment Lifecycle



4.5 Permission Matrix (Draft — We'll Refine)

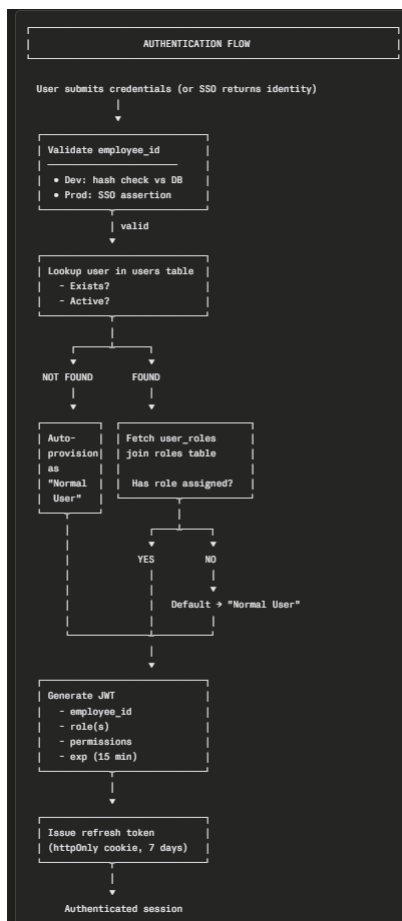
Resource/Action	Normal	View-Only	Lab Engineer	Lab In-charge	Admin	Super Admin
job_request:create	✓	✗	✓	✓	✓	✓
job_request:read (own)	✓	✓	✓	✓	✓	✓
job_request:read (all)	✗	✓	✓	✓	✓	✓
job_request:approve	✗	✗	✗	✓	✓	✓
job_card:execute	✗	✗	✓	✓	✗	✓
job_card:verify/close	✗	✗	✗	✓	✗	✓
equipment:create	✓✗	✗	✗✓	✓	✓	✓
equipment:update	✗	✗	✓	✓	✓	✓
equipment:delete	✗	✗	✗	✗	✗	✓
schedule:manage	✗	✗	✗	✓	✓	✓
procurement:manage	✗	✗	✗	✓	✓	✓
master_data:manage	✗	✗	✗	✗	✓	✓
user_role:assign	✗	✗	✗	✗	✗	✓
reports:view	✓	✓	✓	✓	✓	✓
reports:export	✗	✗	✓	✓	✓	✓
audit_log:read	✗	✗	✗↓	✗	✓	✓

4.4 Why You Need Permissions, Not Just Roles

You said "RBAC" — but proper RBAC has **three layers**, not two:

USER → ROLE → PERMISSION → RESOURCE+ACTION

Layer	Example
User	Employee #EMP1234 (Deep)
Role	Lab Engineer
Permission	job_card:update, equipment:read, report:export
Resource:Action	The granular check the API performs



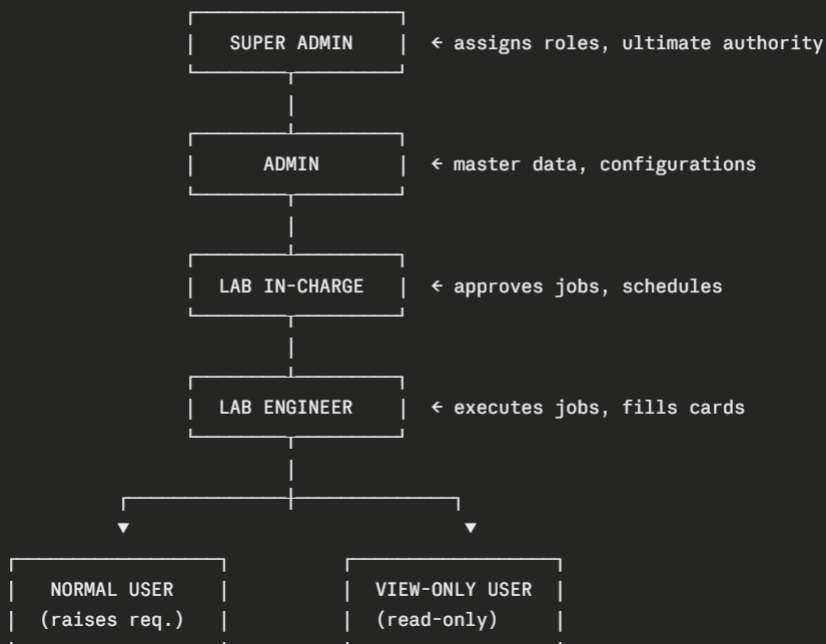
3.2 Module Responsibility Matrix

Module	Owns (CRUD)	Reads From	Writes To	Primary Actors
Dashboard	Nothing	All modules	Nothing	All roles
Job Requests	<code>job_requests</code>	Equipment, Employees	Job Cards (on approval)	Normal User → Lab In-charge
Job Cards	<code>job_cards</code> , <code>tasks</code> , <code>observations</code>	Job Requests, Equipment	Documents, Schedule	Lab Engineer, Lab In-charge
Equipment	<code>equipment</code> , <code>specifications</code> , <code>cal_history</code>	Job Cards, Procurement	Schedule	Lab Engineer, Admin
Schedule	<code>schedules</code> (pm + cal)	Equipment	Notifications	Lab In-charge, Engineer
Procurement	<code>purchase_orders</code> , <code>spare_parts</code>	Vendors, Equipment	—	Admin, Lab In-charge
Inquiry	Nothing (read-only)	Everything	—	All roles
Reports	Nothing (read-only)	All modules	Export logs	Lab In-charge, Admin
Admin	Master data + Users + Roles	—	—	Super Admin, Admin

🔒 Section 4 — Authorization & Role Model (Critical Section)

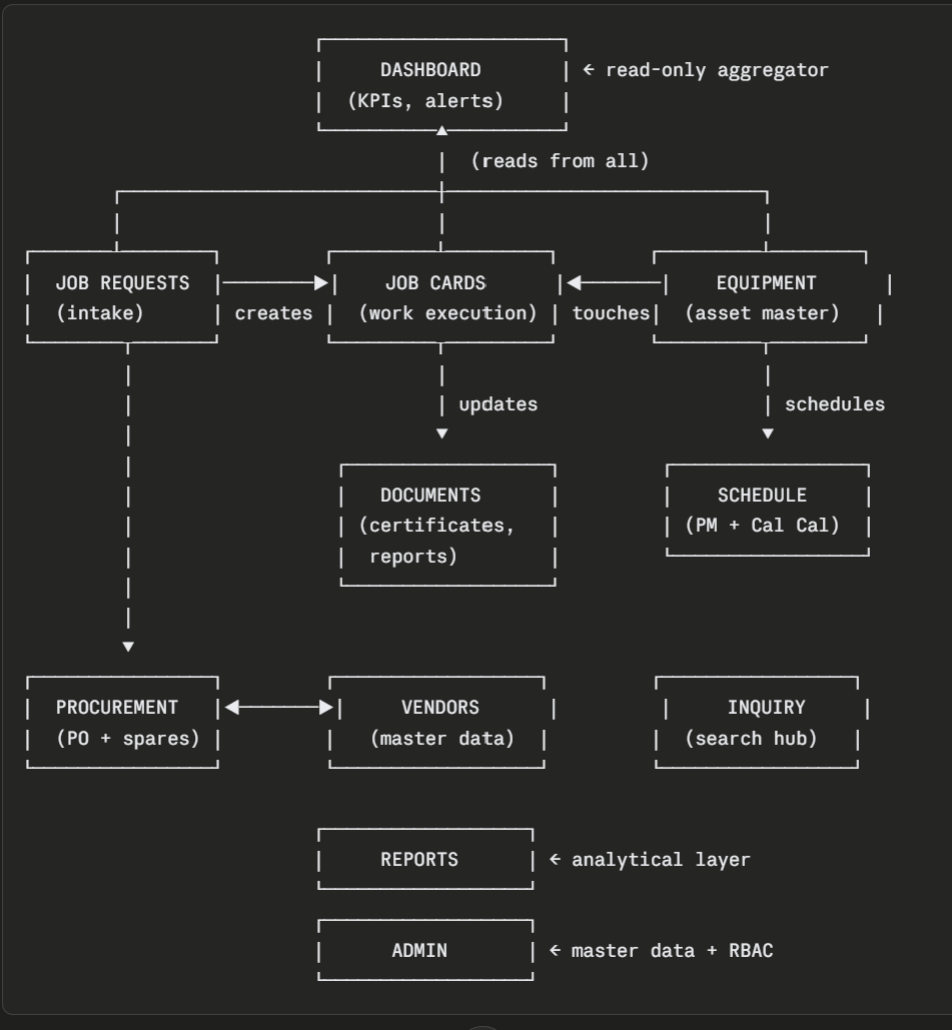
You described the auth flow correctly in spirit, but it has **subtle holes** that will hurt you in production. Let me clean it up.

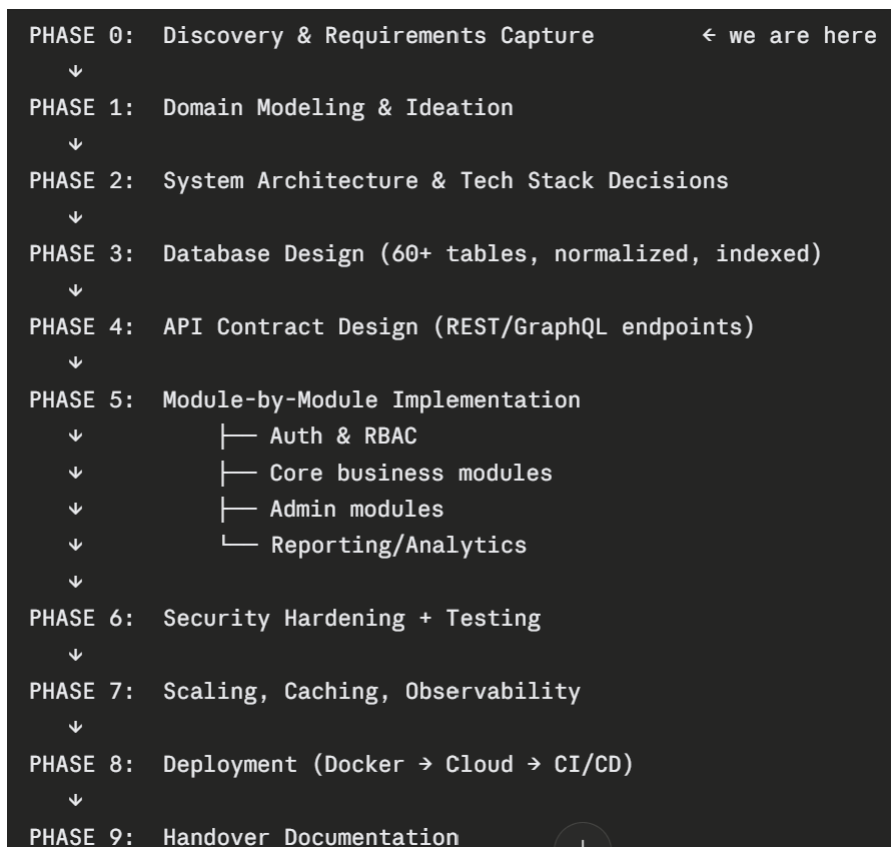
4.1 Role Hierarchy as You Described It



Section 3 — Module Architecture Map

3.1 The 9 Modules and How They Connect





PHASE 0: Discovery & Requirements Capture AND ENTIRE PROJECT DESCRIPTION

i attached here, please take your time and think complex deeper step by step and in depth all the aspects and properly all the view points and give me all the things please take your time and i want response back with my style as always

PLEASE TAKE YOUR TIME

Here is complete CMCMS (Computerized Maintenance & Calibration Management Information System) as per the requirements. Here's what has been implemented:

Complete System Features :-

Global Layout

- Collapsible left sidebar navigation with all 9 modules
- Top header with global search, notifications, and user profile
- Clean, professional design with neutral blue/grey color palette
- Desktop-first, responsive layout

Authorization after SSO login method

1. After successful login with employee id as username and password from as of now from database and at deployment time from organization's SSO as Single Sign On method.
2. Login as normal user and authorized as normal user.
3. After SUPER ADMIN set role of the that normal user and also store that user role into the database.
4. Always check as username or employee id with the database role schemas.
5. Efficient data stored and retrieval and much faster.
6. Database design is much and very important for all the things.

Dashboard (Home)

- Interactive widgets showing pending jobs, calibration due, overdue tasks
- Real-time metrics with visual charts (Recharts)
- Engineer workload bar chart
- Equipment status pie chart
- Calibration due alerts with priority indicators
- Quick actions for creating job requests and adding equipment
- Recently updated jobs table

Job Requests Module

- Comprehensive listing with advanced filters (type, status, search)
- Full-featured form with progressive disclosure

- Multi-section smart form covering:
- Job type selection (T&ME/F&PE, Calibration/Repair/Registration)
- Equipment details with searchable fields
- System type selection
- Conditional procurement details
- Submitted by information (auto-filled)
- Save as draft and submit functionality
- Inline validation and autofill

Job Cards Module

- Job lifecycle tracking with visual status timeline
- Filterable table view with progress indicators
- Detailed job card view with:
- Horizontal status stepper (6 stages)
- Job and equipment information cards
- Interactive task checklist
- Observations and readings log
- Document upload/download section

Equipment Module

- Comprehensive equipment inventory table
- Multi-filter capability (type, status, search)
- Color-coded calibration due dates (overdue alerts)
- Detailed equipment pages showing:
- Full data and specifications
- Calibration status and history timeline
- Procurement and warranty details
- AMC information
- Linked job requests

Schedule Module

- Dual tabs: Preventive Maintenance & Calibration Schedule
- Toggle between calendar and table views
- Interactive calendar with event visualization
- Color-coded events (calibration vs maintenance)
- Scheduling capabilities with status tracking



Procurement Module

- Purchase order tracking table
- Spare parts inventory management
- Low stock alerts (color-coded)
- Vendor linkage
- Dual-tab interface for POs and spare parts



Inquiry Module

- 4-tab search interface:
- Vendor lookup with type filters
- Product catalog search
- Job card status inquiry
- Instrument lookup
- Real-time search across all data
- Comprehensive result tables



Reports Module

- 6 predefined report types with visual selection
- Interactive date range and filter controls
- Visual analytics:
- Monthly job trends (bar chart)
- Division-wise distribution (pie chart)
- Calibration completion trends (line chart)
- Export options (PDF, Excel, Print)

Admin Module

- Master data management with 4 tabs:
- Employees
- Vendors
- Equipment Types
- Divisions
- CRUD operations for all master data
- Role-based access control visualization
- Inline editing and deletion capabilities

Key UX Features

- Maximum 3-click navigation to any feature
- Smart filters throughout
- Inline editing where applicable
- Professional government-grade interface
- Efficient data-first design
- Clear visual hierarchy
- Consistent color coding for status indicators
- Responsive tables with hover states
- Breadcrumb navigation

The system is fully functional as a frontend application with comprehensive data loaded from the database schemas, optimized for technical users (engineers, lab staff, admin) in a high-precision engineering environment like ISRO SAC.

In which we are going to build end to end T&ME and F&PE department calibration & repair for the Instruments & Equipment and also registration of their. with the as now we have user roles like normal users, lab engineer, lab In-charge, Admin and view only user THIS ALL ACT AS NORAML USER BUT AFTER AUTHENTICATION, for

the purpose of role based access control system which is RBAC SYSTEM for this SUPER ADMIN SET ROLE OF THE NORMAL USER AND also this role be stored in the database and for all the user while authentication first of all as match employee id to the database and fetch role from database and if there is not set any role or null values than logged in as normal user and after give super admin as role than store it into database. in this project a "Computerized Maintenance & Calibration Management Information System (CMCMIS)" used in a high-precision engineering environment like ISRO SAC.

The system manages equipment calibration, maintenance, job requests, scheduling, procurement, and reporting. The UI must be extremely efficient, minimal-click, and optimized for technical users (engineers, lab staff, admin).

Answer

🔗 CMCMIS — Phase 0: Discovery & Requirements Capture

I've read the entire description carefully, twice. Before I respond, let me say one honest thing: **this is a genuinely serious project.**

Calibration & maintenance management in a high-precision engineering environment (like ISRO SAC) is not just a CRUD app — it touches **metrological traceability, ISO 17025 compliance, audit trails, and equipment safety**. Some of what you described is right. Some of what you described is missing critical pieces. I'm going to surface both — honestly — so that what we build is actually defensible in a real organization.

Let's go deep.

✦ Section 1 — System Identity (My Understanding in Clear Terms)

Attribute	My Understanding
System name	CMCMIS — Computerized Maintenance & Calibration Management Information System
Domain	Metrology, calibration, maintenance management in high-precision engineering
Reference environment	ISRO SAC-like (space/defense/aerospace lab environment)
Departments served	T&ME (Test & Measurement Equipment) + F&PE (Fabrication & Production Equipment)
Core operations	Calibration · Repair · Registration · Maintenance · Procurement · Reporting
User profile	Technical (engineers, lab staff, admin) — not consumer-grade UX
UX philosophy	Minimal clicks, data-dense, efficient, government/lab-grade
Auth model	SSO at deployment · DB-based credentials during dev · Employee-ID identity
Authorization model	RBAC via Super Admin role assignment, stored in DB
Architecture tier	Industry-grade · Production system · Long-lived (years, not months)
Modules	9 (Dashboard, Job Requests, Job Cards, Equipment, Schedule, Procurement, Inquiry, Reports, Admin)

Section 2 — Domain Crash Course (Why This Shapes Everything)

Before we touch a single table, we must respect the domain.

Calibration is not "just a job ticket." Here's the reality:

2.1 What T&ME and F&PE Actually Mean

Term	Meaning	Example Instruments
T&ME	Test & Measurement Equipment — instruments that <i>measure</i> signals or quantities	Oscilloscopes, multimeters, spectrum analyzers, signal generators, power meters, network analyzers
	Fabrication & Production Equipment — equipment used in <i>making or producing</i> hardware	Soldering stations, vacuum chambers, environmental test chambers, ESD setups, wire bonders, reflow ovens

*****Tech Stack***** Now i am giving you all the technical tech stack frontend ----> react.js JavaScript vite with tailwind css backend - ----> node.js with express.js main database ----> normal SQL as mysql for tool we will going with phpMyAdmin Question: The first time the system goes live, no Super Admin exists in the DB. Who promotes the first Super Admin? Answer===== \ Seeded super admin DB migration inserts one super-admin record (your employee ID, or a configured one) Simple. Organizational Context 1. What is the actual organization name (or just confirm: ISRO/SAC/something similar)? Answer=====

REFER another document named as cmcmis-simplified

FINAL

Josh check: high, sir, high! 🐱

But let me ground that for one second before we sprint. You said *"if I need to sleep late, wake up early, I'm ready."* I appreciate that — but I want to be honest with you up front: **the deciding factor in 10 weeks won't be your hours; it will be our architectural discipline.** A tired solo dev making rushed schema decisions in week 3 will cost us week 8. So our trade is: **work hard, but think first.** That's the only way 10 weeks to production-grade actually happens.

Now — let's build.

📌 Section 1 — Scope Decisions Locked from Phase 0

These are now **non-negotiable** for v1. Anything outside this box goes to backlog.

Decision	Locked-in Choice	Source
Project name	CMCMIS_SIMPLIFIED	Your call
Duration	10 weeks, solo dev	Your call
Domain context	ISRO SAC-like (high-precision engineering, defence/space-grade)	Your call
Auth (v1)	Employee-ID login against existing DB	Your call
Auth (future)	SSO-ready architecture, plug into Active Directory later	Your call
Deployment	On-prem / org private infrastructure	Your call
File storage	Generate-and-download only, no persistence	Your call — <i>big simplifier</i>

Decision	Locked-in Choice	Source
Email/SMTP	Out of scope for v1	Your call
Caching	Optimized SQL + pagination + connection pooling (no Redis)	Your call
Notifications	In-app only (no email/SMS/push)	Inferred
PDFs	Generated on-demand (cal certificates, job cards)	Your call
Barcode/QR	Out of scope	Your call
Mobile/tablet	Desktop/laptop primary, fully responsive viewport	Your call
Multi-language	English only	Inferred
Compliance (NABL/ISO 17025)	Architecture-aware, not feature-enforced (until you say so)	Your call
Sensitive data	Row-level access control, secure sessions, limited exports	Your call
Existing DB	~64 tables already exist; will reconcile in Phase 3	Your call
MVP modules	(1) Auth+RBAC, (2) Equipment+Job Requests+Job Cards, (3) Dashboard+Inquiry	Your call

❑ **Important architectural consequence of "no file storage":** This removes a whole class of complexity. No upload pipeline, no virus scanning, no storage quotas, no backup of files. We **generate PDFs on demand** from DB data and stream them to the user. Documents are **derived state**, not stored state.

❑ **Important architectural consequence of "no email":** No async queues, no notification service. All "notifications" become **in-app records** the user pulls when they log in. Simpler, cleaner.

🕒 **One concern I must flag:** You answered "NO" for backup infrastructure. For a production system at a government-grade org, this is genuinely risky. I'm not going to design backup *now* — but I'm logging it as a **Phase 8 Deployment must-discuss** item. Don't forget.

🔗 Section 2 — Refined Problem Statement

Before (what you described loosely)

"A system to manage equipment calibration, maintenance, job requests, scheduling, procurement, reporting for high-precision lab."

After (what we'll actually build)

CMCMIS_SIMPLIFIED is an internal web-based information system that digitizes the **end-to-end workflow of test/measurement and production equipment** in a high-precision engineering lab — from **registration**, through **calibration and repair job lifecycle**, to **operational visibility** for technical staff. The system replaces ad-hoc tracking (spreadsheets, paper forms, manual follow-ups) with a **role-aware, auditable, single source of truth** that engineers, lab leads, and admins use daily. v1 targets the **three highest-friction workflows** (auth/RBAC, equipment + job lifecycle, dashboard + inquiry) and is architected to extend into scheduling, procurement, and reporting in subsequent phases.

That sentence is what we will deliver. Everything else is in service of it.

3.2 A Day in the Life — Ravi (Lab Engineer)

08:55 Logs in → Dashboard shows: 4 jobs assigned to me, 2 overdue

09:00 Clicks first job card → reads observation history

09:10 Goes to lab → physically calibrates instrument

10:30 Returns → opens job card → enters readings, checks tasks

10:45 Generates PDF cal report → downloads → hands to user

11:00 Marks job "Completed" → moves to next job

...

17:30 Dashboard shows: 2 jobs closed today, 1 in-progress, 1 pending verification

Design implication: Job card view must be the most polished screen in the system. Ravi opens it 10+ times a day.

3.3 A Day in the Life — Meena (Lab In-charge)

09:00 Dashboard → 6 new job requests overnight

09:05 Reviews each → assigns to engineers based on load (sees engineer workload chart)

09:30 Approves 2 high-priority repair requests

10:00 Checks calibration due list → schedules 4 upcoming calibrations

11:00 Verifies 3 completed jobs → marks as closed (or reopens if defect)

14:00 Inquiry: looks up an instrument's full calibration history (auditor question)

Design implication: Assignment UX must be drag-drop or 2-click max. Verification is its own focused screen.

3.4 A Day in the Life — Karthik (Normal User)

10:00 Logs in once a week

10:01 Sees his open job requests + statuses

10:02 Raises new request for an instrument that needs calibration

10:05 Done

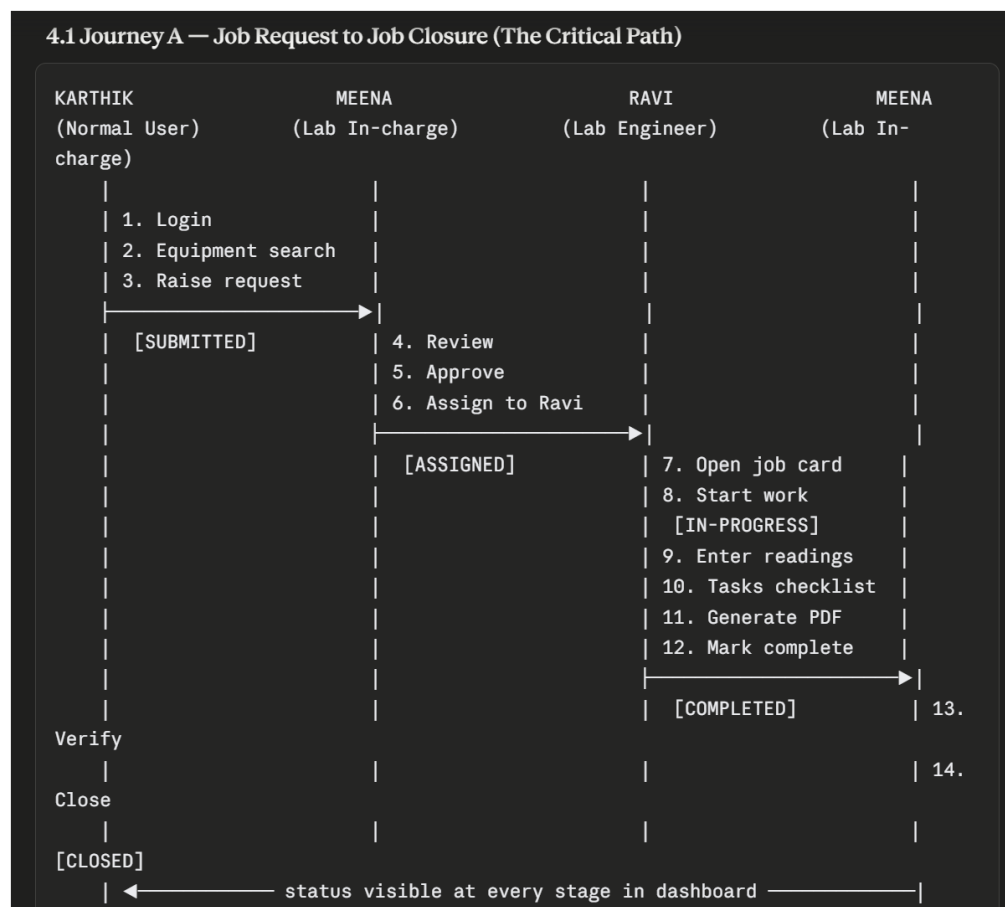
Design implication: Normal-user UX must be **brutally simple**. No clutter. Job request form must be friction-free.



Section 4 — End-to-End User Journeys (MVP Scope)

These three journeys = the entire MVP demo. If these work cleanly, you ship.

4.1 Journey A — Job Request to Job Closure (The Critical Path)



4.2 Journey B — Equipment Registration (Onboarding New Asset)

PRIYA (Admin)

- |
- | 1. Login → Equipment module → "+ Add Equipment"
- | 2. Form: serial no, model, type (T&ME or F&PE), category, manufacturer
- | 3. Specifications: range, accuracy, units
- | 4. Procurement details: PO ref, vendor, date received, warranty until
- | 5. Calibration metadata: frequency (months), last cal date (if existing)
- | 6. System auto-computes: next_cal_due_date
- | 7. Equipment status: ACTIVE
- | 8. Save → equipment appears in master list, available for job requests

4.3 Journey C — Inquiry & Dashboard (Operational Visibility)

ANY USER

- |
- | 1. Login → Dashboard
- | 2. Sees role-filtered KPIs:
 - | └─ My open jobs (Lab Engineer)
 - | └─ Pending approvals (Lab In-charge)
 - | └─ Calibration due in 30 days (everyone)
 - | └─ Overdue alerts (everyone)
- | 3. Click Inquiry → 4-tab interface

- | └— Tab 1: Vendor lookup
- | └— Tab 2: Product/equipment lookup
- | └— Tab 3: Job card status by ID
- | └— Tab 4: Instrument by serial number
- | 4. Result table → drill down to full record

Section 5 — Business Rules Catalogue (Numbered & Traceable)

These rules are **the contract** between the business and the code.
Every API endpoint will enforce them.

5.1 Authentication & Identity Rules

ID	Rule
BR- AUTH-01	Login is by employee_id only (not email/username). Validated against existing employee table.
BR- AUTH-02	A user must exist in the org's employee directory to be allowed to log in. No self-registration.
BR- AUTH-03	If an authenticated user has no role assigned, they default to Normal User .
BR- AUTH-04	Sessions expire after 60 minutes of inactivity. Refresh token valid 7 days.
BR- AUTH-05	The first Super Admin is seeded via DB migration using an employee_id from env config.
BR- AUTH-06	All login attempts (success and failure) are logged.
BR- AUTH-07	A user can be deactivated (status = INACTIVE) — blocks login but preserves history.

5.2 Authorization (RBAC) Rules

ID	Rule
BR-RBAC-01	Only Super Admin can assign or change a user's role.
BR-RBAC-02	A user has exactly one primary role at a time (no multi-role v1 to keep things sane).
BR-RBAC-03	Permissions are derived: User → Role → Permissions. Never checked by role name in code.
BR-RBAC-04	Sidebar items and routes are filtered by permissions, not hardcoded by role.
BR-RBAC-05	Every API endpoint enforces permission check at controller layer.
BR-RBAC-06	View-Only users can read but cannot trigger any write operation.
BR-RBAC-07	Role changes take effect on the user's next login (or token refresh).

5.3 Equipment Rules

ID	Rule
BR-EQP-01	Every equipment has a unique serial number within the system.
BR-EQP-02	Equipment must belong to either T&ME or F&PE category.
BR-EQP-03	Equipment in F&PE may have no calibration frequency (calibration optional).

ID	Rule
-----------	-------------

BR- EQP-04	T&ME equipment must have a calibration frequency (in months).
---------------	---

BR- EQP-05	$\text{next_cal_due_date} = \text{last_cal_date} + \text{calibration_frequency_months}$. Computed on every cal record.
---------------	--

BR- EQP-06	Equipment status transitions follow the equipment state machine (Phase 0 Section 5.2).
---------------	--

BR- EQP-07	Equipment can only be deleted by Super Admin. All others must use status = CONDEMNED.
---------------	---

BR- EQP-08	Equipment search is case-insensitive across serial no, model, manufacturer, type.
---------------	---

5.4 Job Request Rules

ID	Rule
-----------	-------------

BR-JR- 01	A job request must reference an existing piece of equipment (or trigger registration).
--------------	--

BR-JR- 02	Job type $\in \{\text{Calibration, Repair, Registration}\}$.
--------------	---

BR-JR- 03	A user can save a request as DRAFT before submitting.
--------------	---

BR-JR- 04	Once SUBMITTED, only Lab In-charge can change request state.
--------------	--

BR-JR- 05	A submitted request must be assigned to a Lab Engineer before becoming a Job Card.
--------------	--

ID Rule

BR-JR- Auto-fills "submitted by" from current user — cannot be
06 overridden.

BR-JR- Repair requests with high-priority flag appear at top of Lab
07 In-charge's queue.

BR-JR- A request can be rejected with mandatory reason (free text +
08 reason code).

5.5 Job Card Rules

ID Rule

BR- A job card is created **automatically** when a request is
JC-01 approved + assigned.

BR- Job card state follows the lifecycle: ASSIGNED → IN-PROGRESS
JC-02 → COMPLETED → VERIFIED/CLOSED.

BR- Only the assigned engineer can mark a job as IN-PROGRESS or
JC-03 COMPLETED.

BR- Only Lab In-charge can verify/close a job card.
JC-04

BR- A closed job can only be REOPENED by Lab In-charge with
JC-05 reason.

BR- Tasks within a job card are configurable per job type.
JC-06

BR- Calibration job cards require: before-reading, after-reading,
JC-07 environmental conditions before COMPLETED.

BR- Job card history is **append-only** — every state transition is
JC-08 logged immutably.

5.6 PDF Generation Rules

ID	Rule
BR-PDF-01	PDFs are generated on demand from current DB state. Nothing is stored.
BR-PDF-02	Job card PDF contains: header, equipment info, tasks, observations, signatures (textual), date.
BR-PDF-03	Calibration certificate PDF (when applicable): equipment, standards used, readings, environment, uncertainty placeholders, valid-until date.
BR-PDF-04	All PDFs include a generation timestamp + "Generated by CMCMIS" footer + record ID for audit.

5.7 Audit & Security Rules

ID	Rule
BR-AUD-01	Every write operation (INSERT/UPDATE/DELETE) on critical tables is logged in audit_logs.
BR-AUD-02	Audit log captures: who, what, when, before-value, after-value, IP, user-agent.
BR-AUD-03	Exports (PDF, future Excel) are logged with user + record IDs accessed.
BR-AUD-04	Sensitive fields (if any) are filtered out of API responses based on permission.
BR-AUD-05	All API responses use HTTPS in production. JWT in Authorization header, refresh in httpOnly cookie.

5.8 Data Visibility Rules (Row-Level)

ID	Rule
----	------

BR-VIS-01	Normal User sees only their own job requests.
-----------	--

BR-VIS-02	Lab Engineer sees all jobs assigned to them + the queue.
-----------	---

BR-VIS-03	Lab In-charge and above see all jobs and equipment .
-----------	---

BR-VIS-04	View-Only User sees all data, no actions .
-----------	---

Section 6 — Functional Requirements (MVP)

Numbered, traceable. Each maps to API endpoints later.

6.1 Auth & RBAC Module (FR-A)

FR	Description	Priority
FR-A-01	Login screen accepts employee_id + password	MUST
FR-A-02	Successful login returns JWT + sets refresh cookie	MUST
FR-A-03	Logout clears tokens and invalidates session	MUST
FR-A-04	Protected routes redirect to login when unauthenticated	MUST
FR-A-05	Sidebar items render based on user's permission set	MUST
FR-A-06	Super Admin can view list of all users with their roles	MUST

FR	Description	Priority
FR-A-07	Super Admin can assign/change a user's role	MUST
FR-A-08	Super Admin can activate/deactivate a user	MUST
FR-A-09	Backend exposes GET /me returning current user + permissions	MUST
FR-A-10	Token refresh endpoint silently extends session	MUST

6.2 Equipment Module (FR-E)

FR	Description	Priority
FR-E-01	List equipment with pagination, search, filter (type, status)	MUST
FR-E-02	View equipment detail page with specs, cal history, linked jobs	MUST
FR-E-03	Register new equipment (form with validation)	MUST
FR-E-04	Edit equipment details (specs, status)	MUST
FR-E-05	Color-code calibration due dates (green/yellow/red)	MUST
FR-E-06	Show equipment's calibration history timeline	MUST
FR-E-07	Search equipment by serial, model, manufacturer	MUST

FR	Description	Priority
FR-E-08	Soft-delete (status = CONDEMNED) for non-super-admins	MUST

6.3 Job Request Module (FR-JR)

FR	Description	Priority
FR-JR-01	List job requests with filters (type, status, my requests vs all)	MUST
FR-JR-02	Create job request form (multi-section, progressive disclosure)	MUST
FR-JR-03	Save as draft	MUST
FR-JR-04	Submit triggers state transition to SUBMITTED	MUST
FR-JR-05	View request detail with full info + state history	MUST
FR-JR-06	Lab In-charge can approve / reject with reason	MUST
FR-JR-07	Approval triggers automatic Job Card creation	MUST
FR-JR-08	Assignment dropdown shows lab engineers with current workload	SHOULD

6.4 Job Card Module (FR-JC)

FR	Description	Priority
FR-JC-01	List job cards with filter (status, assigned-to-me, all)	MUST

FR	Description	Priority
FR-JC-02	View job card with horizontal status stepper	MUST
FR-JC-03	Task checklist (configurable per job type)	MUST
FR-JC-04	Observations & readings log (free text + structured)	MUST
FR-JC-05	Engineer can transition state (Start work, Mark complete)	MUST
FR-JC-06	Lab In-charge can Verify/Close or Reopen	MUST
FR-JC-07	Generate PDF of job card (on-demand download)	MUST
FR-JC-08	State history visible as timeline	MUST

6.5 Dashboard (FR-D)

FR	Description	Priority
FR-D-01	Role-aware KPI widgets (different for each role)	MUST
FR-D-02	Calibration due alerts (next 30 days + overdue)	MUST
FR-D-03	Engineer workload bar chart (for Lab In-charge)	MUST
FR-D-04	Equipment status pie chart	MUST

FR	Description	Priority
FR-D-05	Recently updated jobs table	MUST
FR-D-06	Quick actions: "Raise request", "Add equipment" (permission-gated)	MUST

6.6 Inquiry (FR-I)

FR	Description	Priority
FR-I-01	4-tab interface (Vendor, Product, Job Card, Instrument)	MUST
FR-I-02	Real-time search with debounce	MUST
FR-I-03	Result tables with drill-down link to full record	MUST
FR-I-04	Empty-state messages when no results	MUST

Section 7 — Non-Functional Requirements

Category	Requirement	Target
Performance	Initial page load (cold)	< 3 sec on intranet
	API response (95th percentile)	< 500 ms
	List pagination	25 items default, 100 max
Security	Password storage	bcrypt, ≥10 rounds

Category	Requirement	Target
	JWT lifetime	15 min
	Refresh token lifetime	7 days, httpOnly cookie, SameSite=Lax
	SQL injection	Parameterized queries everywhere
	XSS	React auto-escaping + Content-Security-Policy header
	CSRF	SameSite cookies + CSRF tokens on state-changing endpoints
Reliability	Server uptime target (post go-live)	99% during business hours
	DB connection pool	10-20 connections
Usability	Max clicks to any feature	≤ 3
	Keyboard shortcuts	Available on power-user screens
	Responsive viewport	1280px–1920px primary; degrades gracefully to 768px
Audit	Every state-changing operation logged	Yes
	Audit log retention	Indefinite (until you specify)
Maintainability	Code style	ESLint + Prettier enforced

Category	Requirement	Target
	Folder structure	Strict separation: routes / controllers / services / models / utils
	API versioning	/api/v1/... from day 1
Compatibility	Browsers	Chrome, Edge, Firefox (latest 2 versions)

📄 Section 12 — Summary Notes

Topic	Locked Decision
Project name	CMCMIS_SIMPLIFIED
Timeline	10 weeks, solo dev
MVP	Auth+RBAC · Equipment+Jobs · Dashboard+Inquiry
Out of MVP	Schedule · Procurement · Reports · Admin master data · SSO · Email · QR
Auth v1	Employee-ID + DB password; SSO-ready architecture
Storage	DB only (no file storage; PDFs generated on demand)
Notifications	In-app only, no email

Topic	Locked Decision
Personas defined	6 roles with day-in-life workflows
Business rules captured	40+ rules across 8 domains
Functional requirements	40+ FRs across 6 modules
NFRs	Performance, security, audit, usability all explicit
Risk register	7 active risks, mitigations defined
Next phase	Phase 2 — Architecture & Tech Stack

✓ Decision Point — Your Move

Before we move to Phase 2, I need three things from you:

1. **Sign-off:** Do you agree with the MVP scope split in Section 8? Anything you want moved IN or OUT?
2. **Personas check:** Are the 6 personas (Ravi, Meena, Karthik, Priya, Suresh, Anjali) realistic for your org? Any roles missing or extra?
3. **Timeline reality check:** Look at the Week 1–10 plan in Section 9. Does anything feel impossible or too easy? Now is the time to push back.

Answer those three, and we go to Phase 2. 🚀

Josh check from my side: locked in. Let's build something real. 🤝

• Sign-off: Do you agree with the MVP scope split in Section 8? Anything you want moved IN or OUT? Answer===== \yes I agreed with this Scope Modules / Features MVP (10 weeks) Auth + RBAC · Equipment master + register · Job Requests · Job Cards · Dashboard · Inquiry · PDF generation · Audit logs (basic) · Responsive UI Phase 2 (post-internship handoff) Schedule module (PM + Cal calendar) · Procurement (POs + spares) · Reports (analytics + exports) · Admin (master data CRUD) · Notifications (in-app feed) • Personas check: Are the 6 personas (Ravi, Meena, Karthik, Priya, Suresh, Anjali) realistic for your org? Any roles missing or extra? Answer===== \all things are okay but there is small correction as rather than viewer role any of the roles as super admin, admin, lab in-charge, lab engineer and normal user CAN be able to REGISTERED A NEW EQUIPMENTS. • Timeline reality check: Look at the Week 1–10 plan in Section 9. Does anything feel impossible or too easy? Now is the time to push back. Answer===== \okay I will give you instructions for the same.

GREEN FLAG

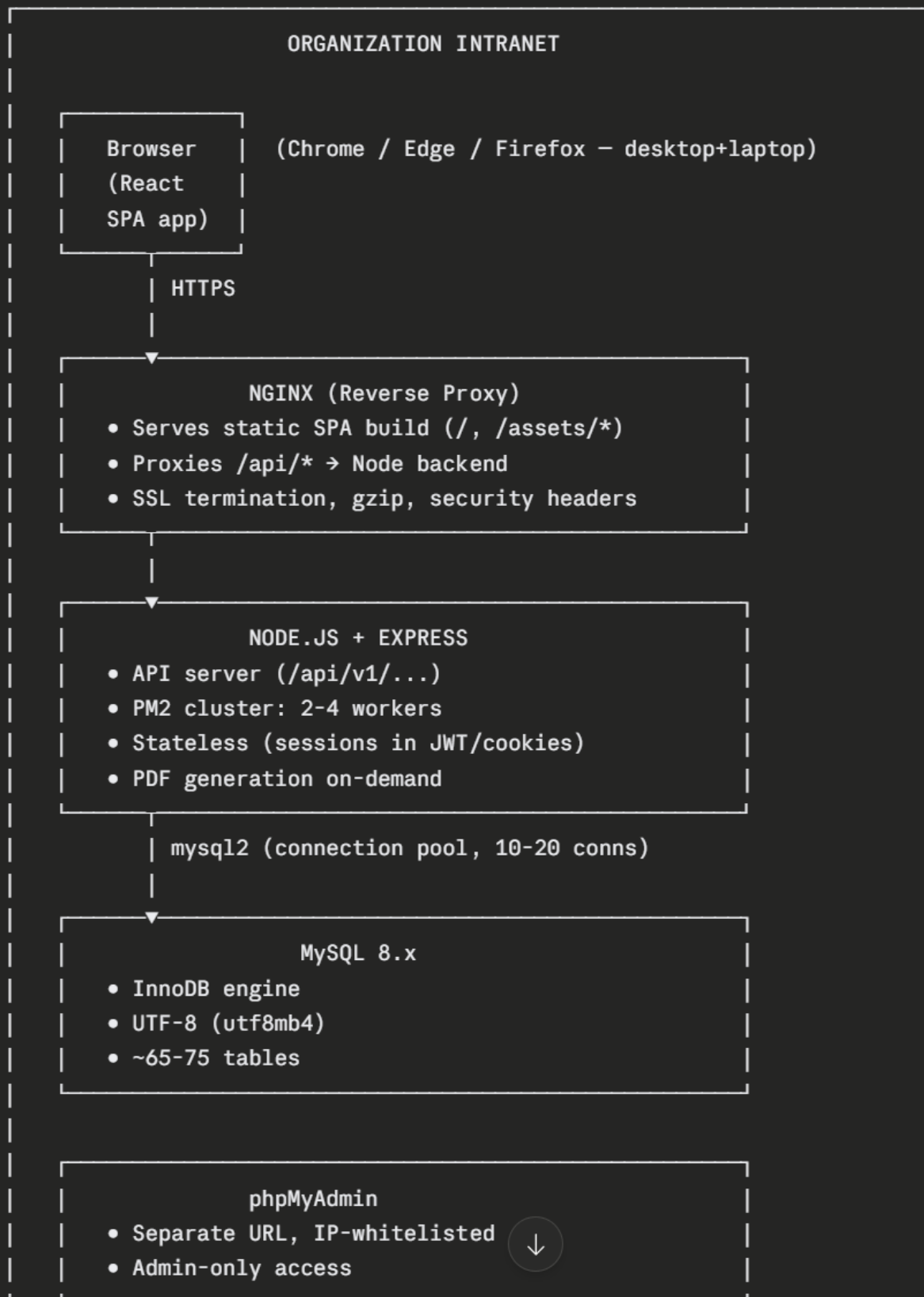
Now — Phase 2

The next 12 sections define the **technical backbone** of CMCMIS_SIMPLIFIED. Every decision here is justified, and every justification ties back to a constraint you gave me. No fashionable picks. No over-engineering. No under-engineering.



Section 1 — High-Level System Architecture

1.1 The Big Picture (One Diagram)



CMC MIS_SIMPLIFIED — Phase 2: System Architecture & Tech Stack

GREEN FLAG received.  Moving in.

But first — one quick correction from your persona feedback. It's a real RBAC change, not cosmetic.

Section 0 — Update from Phase 1 (Persona Correction)

You said: *"any role except View-Only can register a new equipment."*

That changes one row in our permission matrix and one functional requirement. Locking it in:

Resource/Action	Normal	View-Only	Lab Engineer	Lab In-charge	Admin	Super Admin
equipment:create	 (updated)		 (updated)			
equipment:update						
equipment:delete						

One honest note + a safeguard I'm adding: Letting any user register equipment is unusual in metrology — normally it's a controlled master record. We won't block it (your org, your rule), but I'm adding **two safeguards** to keep the system clean:

Safeguard	What It Does	Rule ID
Every registration carries registered_by (employee_id) + timestamp	Full provenance for audit	BR-EQP-09 (new)

Safeguard	What It Does	Rule ID
New equipment defaults to status PENDING_VERIFICATION until Lab In-charge or above flips to ACTIVE	Prevents junk records polluting active operations	BR-EQP-10 (new)

This way you keep openness AND data quality. If you want this stricter (e.g., direct ACTIVE on create), tell me and I'll drop BR-EQP-10. Otherwise, locked in.

Section 3 — Backend Folder Structure

3.1 The Structure

```
backend/
├─ src/
│  ├─ config/
│  │  ├─ env.js           # loads + validates env vars (boot-time fail)
│  │  ├─ db.js           # mysql2 connection pool factory
│  │  ├─ constants.js    # role names, statuses, magic numbers
│  │  └─ logger.js       # pino logger instance
│  │
│  ├─ db/
│  │  ├─ migrations/    # versioned SQL files (001_init.sql ...)
│  │  └─ seeders/       # JS scripts to seed roles, permissions, super
│  │
│  └─ admin
│     ├─ migrate.js      # migration runner CLI
│     │
│     └─ routes/
│        └─ v1/
│           ├─ index.js   # mounts all v1 routes
│           ├─ auth.routes.js
│           ├─ users.routes.js
│           ├─ equipment.routes.js
│           ├─ jobRequests.routes.js
│           ├─ jobCards.routes.js
│           ├─ dashboard.routes.js
│           └─ inquiry.routes.js
│
│     └─ controllers/    # HTTP layer (parse req → call service →
format res)
│        ├─ auth.controller.js
│        ├─ equipment.controller.js
│        └─ ...
│
│     └─ services/       # business logic (the brain)
│        ├─ auth.service.js
│        ├─ equipment.service.js
│        ├─ jobRequest.service.js
│        ├─ jobCard.service.js
│        └─ audit.service.js
│
│     └─ repositories/   # raw SQL queries (data access only)
│        ├─ user.repo.js
│        ├─ equipment.repo.js
│        ├─ jobRequest.repo.js
│        └─ jobCard.repo.js
│
│     └─ middleware/
│        ├─ auth.middleware.js    # JWT verification
│        ├─ permissions.middleware.js # checks resource:action
│        ├─ error.middleware.js   # central error handler
│        ├─ requestLogger.middleware.js
│        ├─ ratelimit.middleware.js
│        └─ validate.middleware.js # r ↓ zod schemas
```

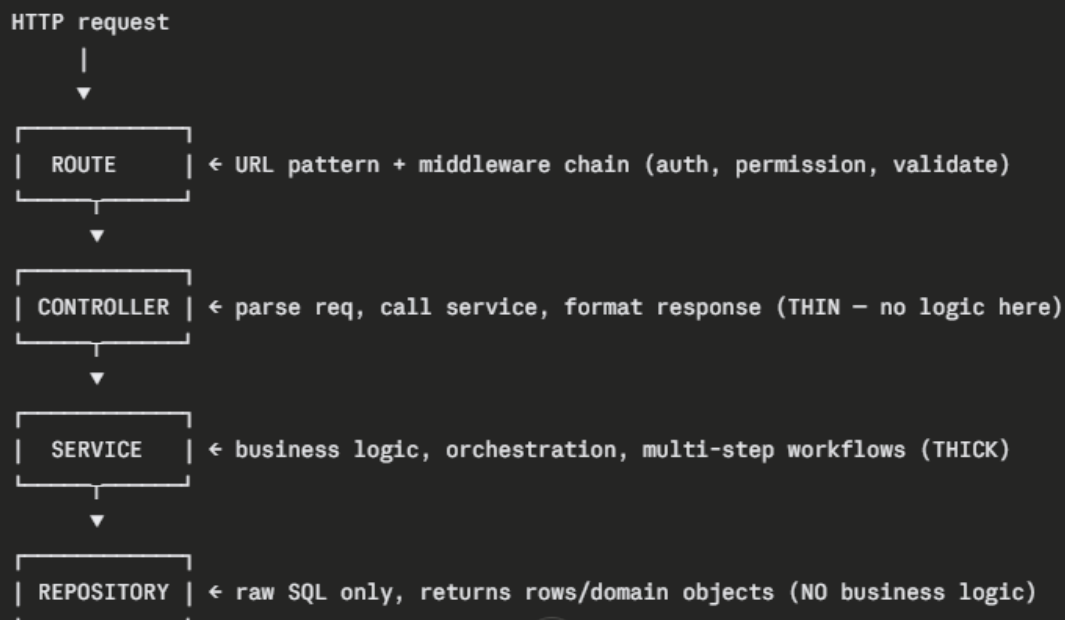


```

├── validators/                # zod schemas
│   ├── auth.validator.js
│   ├── equipment.validator.js
│   └── ...
├── errors/
│   ├── ApiError.js           # base error class
│   ├── ValidationError.js
│   ├── NotFoundError.js
│   ├── ForbiddenError.js
│   └── ConflictError.js
├── pdf/
│   ├── pdfGenerator.js       # base pdfkit wrapper
│   ├── jobCardPdf.js         # job card layout
│   └── calibrationCertPdf.js # cert layout
├── utils/
│   ├── date.js               # dayjs wrappers
│   ├── pagination.js
│   ├── apiResponse.js        # success/error response envelope helpers
│   └── hash.js                # bcrypt wrappers
├── app.js                     # express app: middleware → routes → error
├── server.js                  # entry point (loads env, starts app)
├── ecosystem.config.js        # PM2 config
├── package.json
├── .env.example
├── .eslintrc.cjs
├── .prettierrc
├── .gitignore
└── README.md

```

3.2 Why This Layering? (Routes → Controllers → Services → Repositories)



MySQL

Layer	Owns	Does NOT Own
Route	URL → handler mapping, middleware chain	Business logic
Controller	HTTP req/res shaping	SQL, business rules
Service	Business rules, transactions, state machines	HTTP concerns, raw SQL
Repository	SQL queries, parameter binding	Business logic, validation

Why this matters: When you want to add a CLI script that registers equipment without going through HTTP, you call the **service**, not the controller. Layers stay swappable. This is the difference between code that lives 5 years vs code rewritten in year 2.

📁 Section 4 — Frontend Folder Structure

4.1 The Structure (Feature-Based, Not Layer-Based)

```

frontend/
├── src/
│   ├── api/
│   │   ├── client.js           # axios instance + interceptors
│   │   ├── auth.api.js
│   │   ├── equipment.api.js
│   │   └── ...
│   ├── components/
│   │   ├── ui/                 # primitive, reusable, unstyled-or-tailwind-
only
│   │   │   ├── Button.jsx
│   │   │   ├── Input.jsx
│   │   │   ├── Select.jsx
│   │   │   ├── Modal.jsx
│   │   │   ├── Table.jsx
│   │   │   ├── Pagination.jsx
│   │   │   ├── Badge.jsx
│   │   │   ├── Spinner.jsx
│   │   │   └── EmptyState.jsx
│   │   ├── layout/
│   │   │   ├── AppShell.jsx    # top header + sidebar + outlet
│   │   │   ├── Sidebar.jsx
│   │   │   ├── Header.jsx
│   │   │   └── Breadcrumbs.jsx
│   │   └── shared/
│   │       ├── StatusBadge.jsx # job/equipment status colors
│   │       ├── KpiCard.jsx
│   │       ├── DataTable.jsx   # generic table w/ pagination
│   │       └── PermissionGate.jsx # wraps children, hides if no permission
│   ├── features/
│   │   ├── auth/
│   │   │   ├── pages/
│   │   │   │   └── LoginPage.jsx
│   │   │   ├── hooks/
│   │   │   │   └── useLogin.js
│   │   │   └── store/
│   │   │       └── authStore.js # zustand
│   │   ├── equipment/
│   │   │   ├── pages/
│   │   │   │   ├── EquipmentListPage.jsx
│   │   │   │   ├── EquipmentDetailPage.jsx
│   │   │   │   └── EquipmentRegisterPage.jsx
│   │   │   ├── components/
│   │   │   │   ├── EquipmentTable.jsx
│   │   │   │   ├── EquipmentFilters.jsx
│   │   │   │   └── CalibrationHistoryTimeline.jsx
│   │   │   └── hooks/
│   │   │       └── useEquipment.js
│   │   ├── jobRequests/
│   │   └── jobCards/

```



```

├── jobRequests/
├── jobCards/
├── dashboard/
├── inquiry/
├── hooks/ # cross-feature hooks
│   ├── useAuth.js
│   ├── usePermissions.js
│   └── useDebounce.js
├── store/ # global zustand stores (if any)
│   └── uiStore.js # sidebar collapsed, theme, toasts
├── routes/
│   ├── AppRouter.jsx # main router
│   ├── ProtectedRoute.jsx # auth guard
│   └── PermissionRoute.jsx # permission guard
├── utils/
│   ├── date.js
│   ├── format.js
│   └── classNames.js # cn() helper
├── constants/
│   ├── statuses.js # status enums + colors
│   ├── roles.js
│   └── permissions.js
├── styles/
│   └── globals.css # tailwind + base styles
├── App.jsx
├── main.jsx
├── public/
├── index.html
├── vite.config.js
├── tailwind.config.js
├── postcss.config.js
├── package.json
├── .env.example
├── .eslintrc.cjs
├── .prettierrc
├── .gitignore
└── README.md

```

5.3 HTTP Status Code Discipline

Status	When
200	Successful GET, PUT, PATCH
201	Successful POST creating a resource
204	Successful DELETE (no body)
400	Validation error
401	Not authenticated (no/invalid/expired token)
403	Authenticated but lacks permission
404	Resource not found
409	Conflict (duplicate serial, invalid state transition)
422	Semantically invalid (rare)
500	Unexpected server error



5.4 Query Conventions

Concern	Convention
Pagination	?page=1&pageSize=25 (default pageSize = 25, max = 100)
Sorting	?sortBy=createdAt&sortDir=desc
Filtering	?status=active&type=tme
Search	?q=oscilloscope (free-text on indexed columns)
Field projection	?fields=id,serialNumber,model (future, not v1)

Section 6 — Library Stack Decisions

6.1 Backend Stack

Concern	Library	Version	Why
Web framework	express	^4.x	Stable, ubiquitous, simple. Hapi/Fastify offer little here.
DB driver	mysql2/promise	^3.x	Fastest MySQL driver in Node, native promises
Validation	zod	^3.x	Single source of truth for schemas; replaces Joi/Yup
Auth tokens	jsonwebtoken	^9.x	Industry standard
Password hash	bcryptjs	^2.x	Pure-JS bcrypt, no native build pain

Concern	Library	Version	Why
Logger	pino + pino-pretty (dev)	^8.x	Faster than Winston, structured JSON
Env loader	dotenv	^16.x	Standard
Env validation	envalid	^8.x	Fail at boot if env missing
Date	dayjs	^1.x	Lightweight (2KB), moment-compatible API
PDF	pdfkit	^0.14.x	Programmatic, lightweight, server-friendly
Rate limit	express-rate-limit	^7.x	Simple, in-memory; sufficient for v1
Security headers	helmet	^7.x	OWASP-friendly defaults
CORS	cors	^2.x	Standard
Process mgr	pm2	latest	Cluster mode, restarts, logs
Lint/Format	eslint + prettier + husky + lint-staged	latest	Enforced at commit

6.2 Frontend Stack

Concern	Library	Why
Build	vite	Fast HMR, modern, Vite + React is the 2026 default
Framework	react 18	You're already on it

Concern	Library	Why
Routing	react-router-dom v6	Standard SPA routing
State (local)	React useState/useReducer	Built-in
State (global)	zustand	Tiny (~1KB), no boilerplate, scales to ~50 stores
Server state	@tanstack/react-query	Caching, refetch, mutations — saves 30% of FE code
HTTP	axios	Interceptors for auth, error handling
Forms	react-hook-form + zod	Performant, schema-shared with backend
UI primitives	Custom on top of Tailwind	Avoids design lock-in of shadcn/MUI
Icons	lucide-react	Clean, tree-shakable
Charts	recharts	You mentioned it, fine choice
Date	dayjs	Same as backend, code symmetry
Notifications (toast)	sonner	Lightweight, beautiful defaults
Styling	tailwindcss v3	Already chosen
Lint/Format	eslint + prettier + husky	Same as backend

Express.js is primarily a web framework, not just a normal library.

Simple Understanding

Thing	Meaning
Library	A collection of functions/tools you call when needed
Framework	A complete structure that controls application flow and helps build applications faster

Express gives you:

- Routing system
- Middleware handling
- Request/response management
- API creation tools
- Server architecture support

So it acts as a minimal and flexible web framework for Node.js.

Relationship Between Node.js and Express

Without Express (Only Node.js)

You manually handle everything:

```
const http = require("http");
```

```
const server = http.createServer((req, res) => {
```

```
    if (req.url === '/') {  
      res.end("Home Page");  
    }  
  });  
  
server.listen(3000);
```

With Express

Much cleaner and easier:

```
const express = require("express");  
const app = express();  
  
app.get("/", (req, res) => {  
  res.send("Home Page");  
});  
  
app.listen(3000);
```

Technically Speaking

Express is:

- Built on top of Node.js
 - Uses Node's http module internally
 - Provides abstraction and structure
 - Follows middleware architecture
-

Final Classification

Technology Type

Node.js Runtime Environment

Express.js Web Framework

React Frontend Library

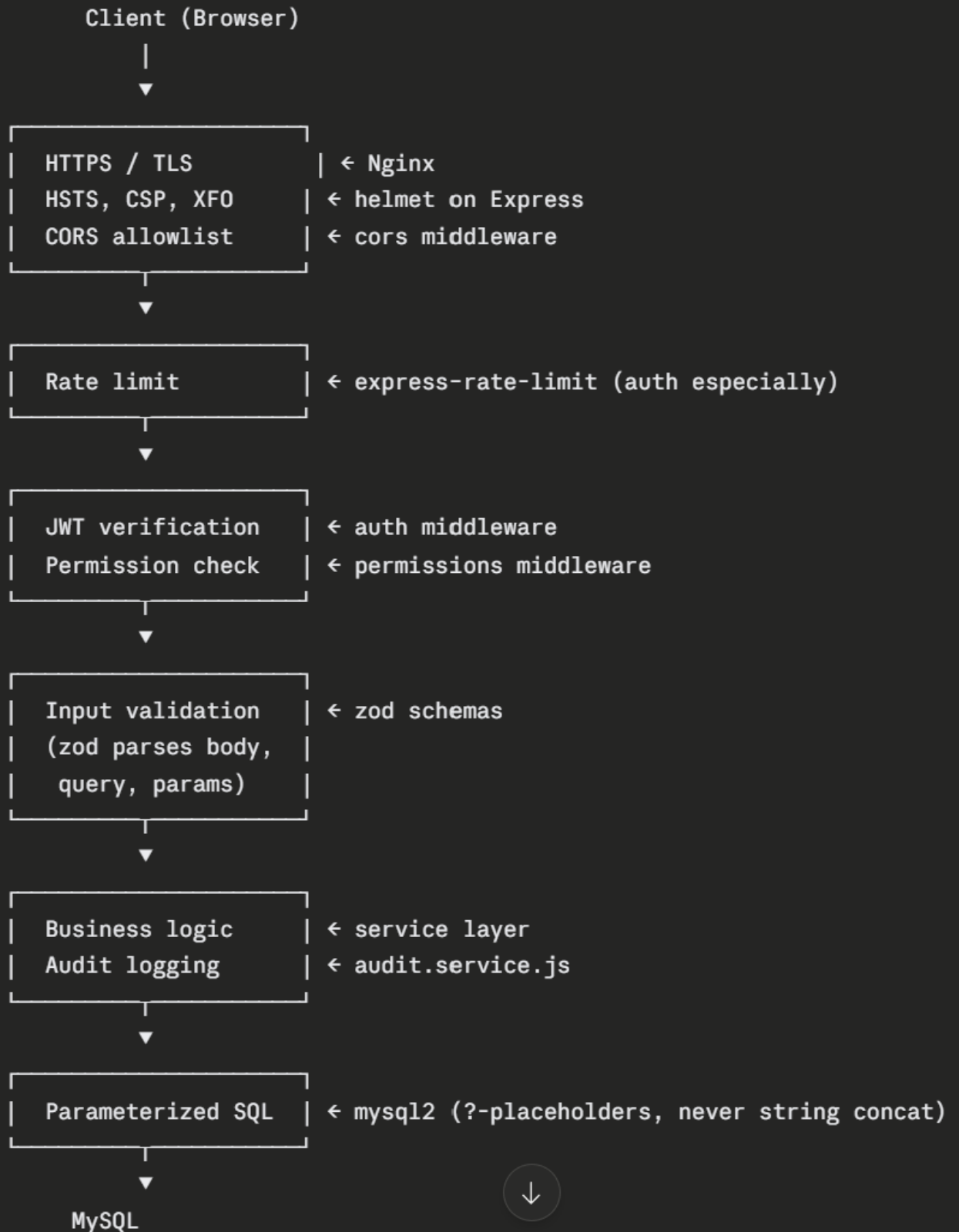
Next.js Full-stack Framework

So:

Express is a backend web application framework for Node.js.

Section 8 — Security Architecture

8.1 Security Layer Stack



10.3 Custom Error Classes

ApiError (base)

└─ ValidationError (400)

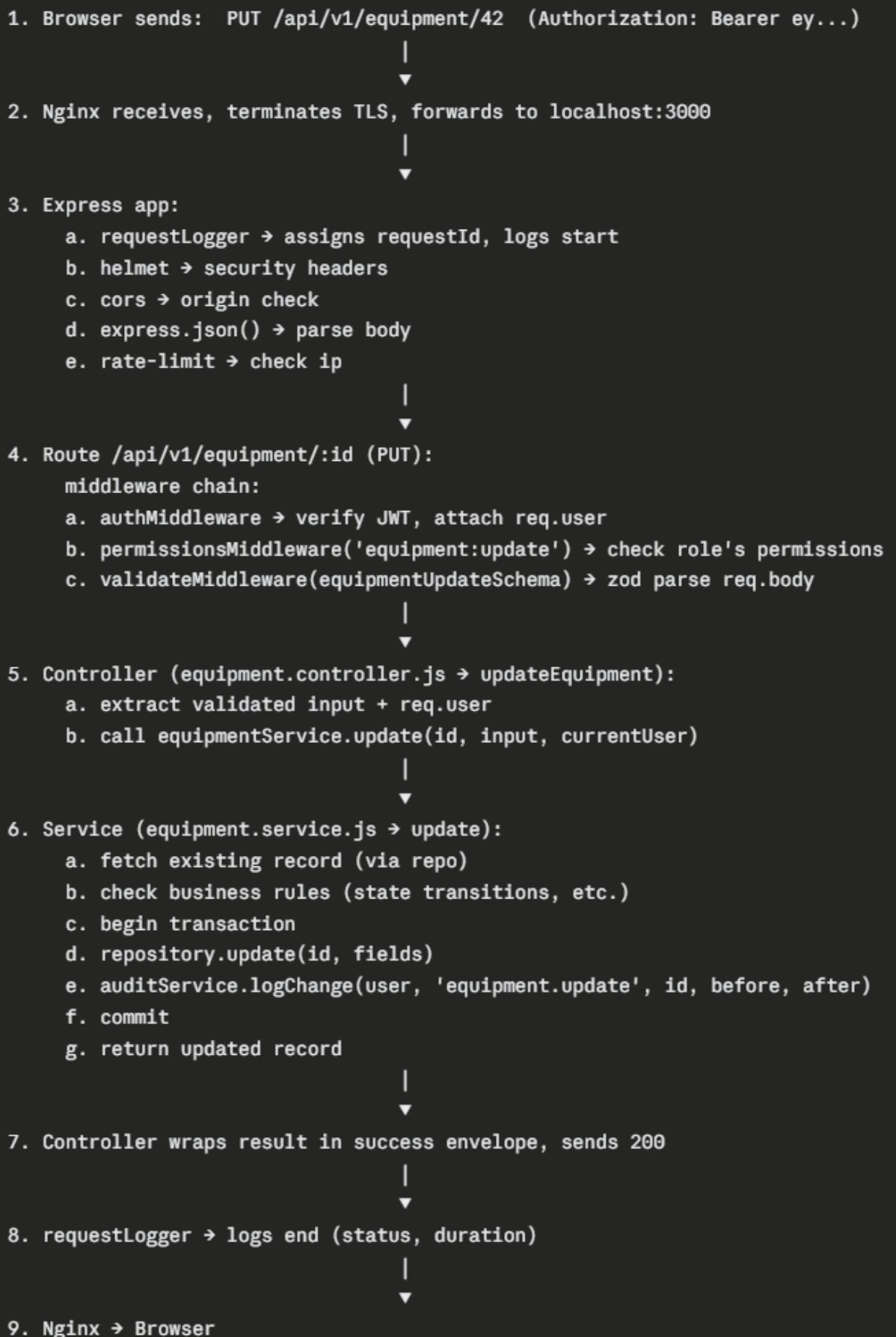
└─ UnauthorizedError (401)

└─ ForbiddenError (403)

└─ NotFoundError (404)

└─ ConflictError (409)

Section 12 — Request Lifecycle (End-to-End Mental Model)



*******IMPORTANT*******

Section 14 — Decisions I Need You to Confirm

#	Decision	My Recommendation	Confirm?
D1	JavaScript or TypeScript?	JavaScript + JSDoc + Zod	YES
D2	ORM vs Raw SQL?	Raw SQL + Repository pattern	YES
D3	Add React Query?	Yes (saves massive FE code)	☐ YES
D4	Zustand for state?	Yes (over Redux)	☐ YES
D5	pdfkit for PDFs?	Yes (over Puppeteer)	☐ YES
D6	Pino for logging?	Yes (over Winston)	☐ YES
D7	Repository pattern in BE?	Yes (layer routes → controllers → services → repos)	☐ YES
D8	Feature-based FE folders?	Yes (over layer-based)	☐ YES
D9	Nginx reverse proxy in production?	Yes (standard)	☐ YES

#	Decision	My Recommendation	Confirm?
D10	New equipment defaults to PENDING_VERIFICATION?	Yes (safety from Section 0)	☐ YES

Section 9 — Phase 2 Decisions Update

Tiny update to one decision from last message's Section 14:

#	Decision	Status
D11 (new)	Seed at least 2 Super Admin employee IDs in SUPER_ADMIN_EMPLOYEE_IDS env var	Recommended ☒

ANSWER-----→yes we have to seed 2 super admin employee ID I will provide you tomorrow this 2 ids while developing or design database schemas.

What I need from you to begin Phase 3:

- Export the existing DB schema (just SHOW CREATE TABLE outputs for all 64 tables, or mysqldump --no-data).

ANSWER-----→ tomorrow at early morning I will provide this details please keep it tomorrow.

- A short note on which existing tables you know are **definitely still used**, which are **probably dead**, and which you're **unsure** about. ANSWER-----→ tomorrow at early morning I will provide this details please keep it tomorrow.

ONCE I WILL PROVIDE YOU TOMMOROW AT EARLY MORNING
LIKE existing database schemas or tables and than we will
edit and design new schemas from scratch or from the basics
for the database. The deepest, most important phase of all.

**Energy level: locked. Discipline level: locked. Let's keep
moving. 🚀**

✓ Section 11 — Confirmation Points

Confirm?

Five roles final: Super Admin, Lab In-charge, Lab Engineer,
C1 Normal User, View-Only

ANSWER-----→ YES

Master Data Management remains Phase 2, accessible by
C2 Super Admin only

**ANSWER-----→ I WILL PROVIDE YOU INSTRUCTIONS
FOR THE SAME WHILE WE DEVELOP IT.**

Lookup data (vendors, equipment types, divisions) will be
seeded in Week 2 and edited via phpMyAdmin during
MVP

ANSWER-----→YES

Confirm?

Bootstrap with at least 2 Super Admin employee IDs
C4 (you'll provide them when we get to seeding)

ANSWER-----→YES

Equipment verification (PENDING → ACTIVE) belongs to
C5 Lab In-charge + Super Admin

ANSWER-----→YES

Locked in. 5 roles. Clean model. Moving.  

☐ Section 16 — Phase 2 Summary

Topic	Locked-In Decision
Architecture	3-tier monolith (SPA + Express API + MySQL)
Deployment	Single Linux server, Nginx + PM2 + MySQL
Backend layers	Routes → Controllers → Services → Repositories
Frontend org	Feature-based, not layer-based
API base	/api/v1/... with standard success/error envelope
Auth	JWT (15min) + refresh cookie (7d, httpOnly)
Bootstrap	Super admin seeded via env config
DB access	Raw SQL via mysql2/promise + repository pattern

Topic	Locked-In Decision
Validation	Zod schemas at controller boundary
FE state	Zustand + React Query + Context (for auth)
Forms	React Hook Form + Zod
PDF	pdfkit, streamed, no storage
Logging	Pino, structured JSON, request-ID correlated
Security	helmet + cors + rate-limit + bcrypt + parameterized SQL
Env config	dotenv + envalid, fail-at-boot
Equipment safety	New equipment → PENDING_VERIFICATION until verified
Audit logs	Every write operation, async, fire-and-forget

Permission Matrix ----- please consider this and the major changes is that we have only 5 role now PLEASE DELETE ALL THE CONTEXT RELATED TO THE ADMIN, in our system we kept only one as SUPER ADMIN. But there is small correction as

rather than viewer role any of the roles as super admin, lab in-charge, lab engineer and normal user CAN be able to REGISTERED A NEW EQUIPMENTS.

please revise all the things

 CMCMIS_SIMPLIFIED — Revision Pack: 5-Role Restructure

Got it. Important change — locking it in. This isn't cosmetic; it ripples through the permission model, personas, modules, and a few business rules. Let me consolidate everything affected in one clean document so nothing drifts.

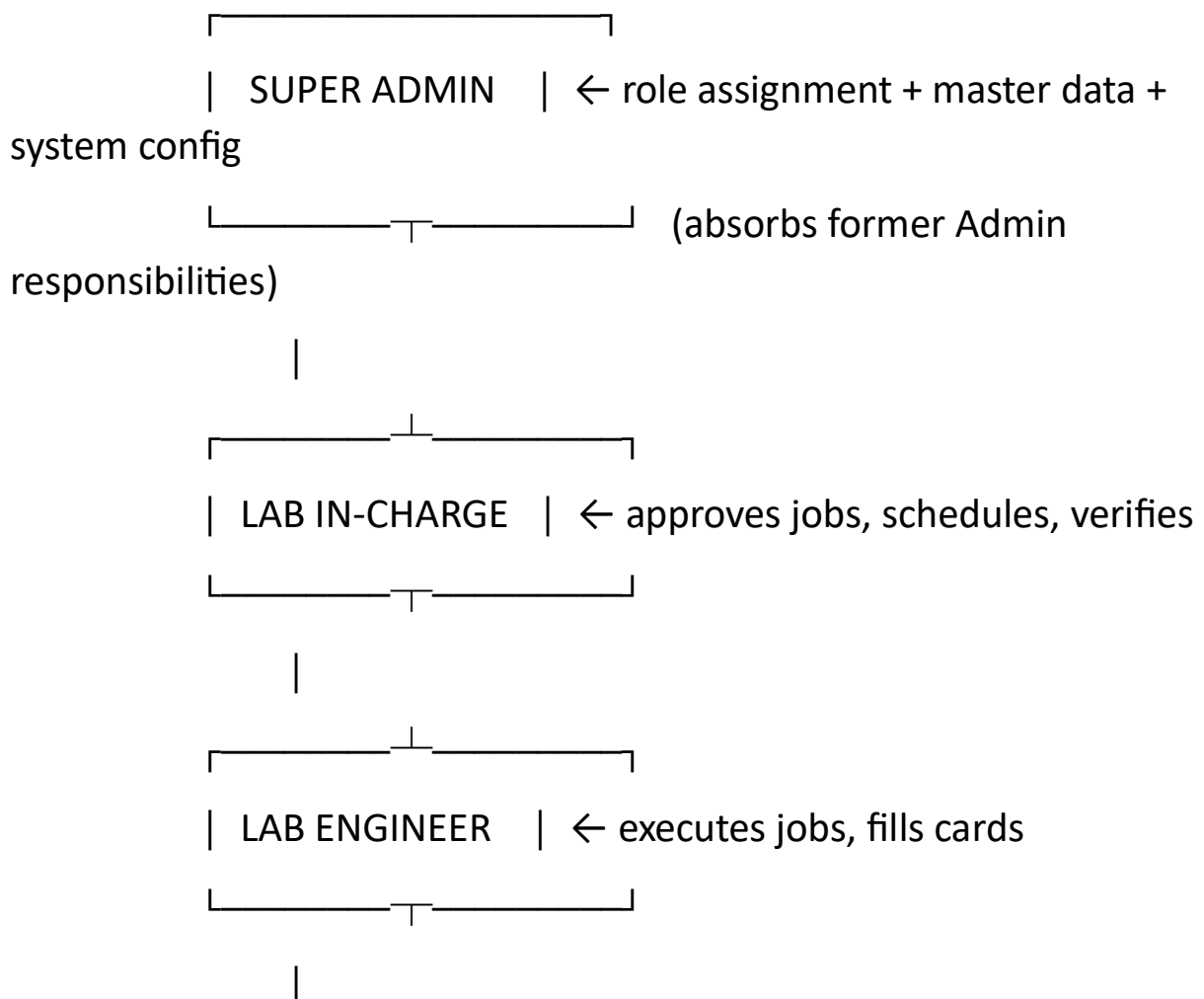
 Section 1 — Summary of the Change

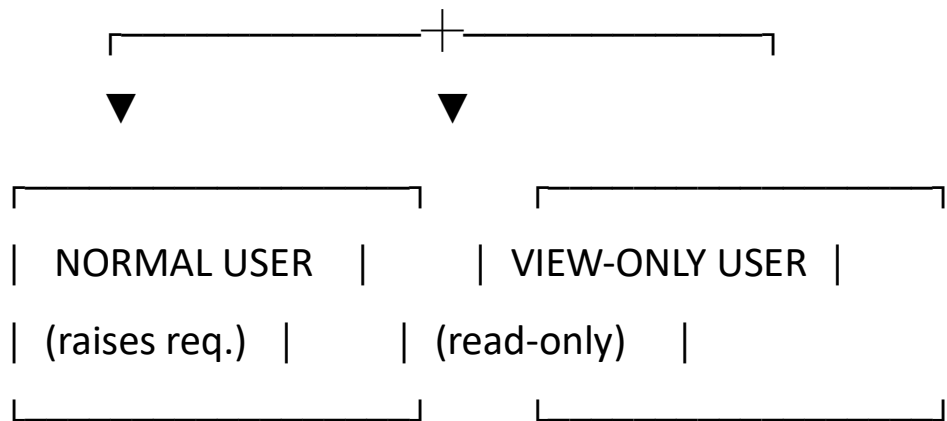
Aspect	Before	After
Total roles	6	5
Roles	Super Admin, Admin, Lab In-charge, Lab Engineer, Normal User, View-Only	Super Admin, Lab In-charge, Lab Engineer, Normal User, View-Only
Personas	6 (Ravi, Meena, Karthik, Priya, Suresh, Anjali)	5 (Priya removed — Admin gone)
Master data ownership	Admin role	Super Admin (absorbs all Admin responsibilities)

Aspect	Before	After
Equipment registration permission	Admin + 4 others	Super Admin + Lab In-charge + Lab Engineer + Normal User (4 roles, no View-Only)

One honest note before we proceed: Concentrating role-assignment + master-data + system-config all on Super Admin means that role is now very powerful. For a small lab this is fine. Just be aware: Super Admin can't be one overworked person — there should be at least 2 employee IDs seeded as Super Admin so coverage exists during leave/handover. I'll bake this assumption in.

Section 2 — New Role Hierarchy (Updated Diagram)





One administrative axis now, not two. Cleaner.

Section 3 — Final Permission Matrix (Clean Version)

This **replaces** all previous permission matrices.

3.1 Authentication & Identity

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
auth:login	✓	✓	✓	✓	✓
auth:logout	✓	✓	✓	✓	✓
auth:refresh-token	✓	✓	✓	✓	✓
me:read (own profile)	✓	✓	✓	✓	✓

3.2 User & Role Management

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
user:read-list	✗	✗	✗	✗	✓
user:role-assign	✗	✗	✗	✗	✓

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
user:activate / user:deactivate	✗	✗	✗	✗	✓

3.3 Equipment

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
equipment:read-list	✓	✓	✓	✓	✓
equipment:read-detail	✓	✓	✓	✓	✓
equipment:create ✨	✓	✗	✓	✓	✓
equipment:update	✗	✗	✓	✓	✓
equipment:verify (PENDING → ACTIVE)	✗	✗	✗	✓	✓
equipment:delete (hard)	✗	✗	✗	✗	✓
equipment:condemn (status → CONDEMNED)	✗	✗	✗	✓	✓

3.4 Job Requests

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
job_request:create	✓	✗	✓	✓	✓
job_request:read-own	✓	✓	✓	✓	✓
job_request:read-all	✗	✓	✓	✓	✓
job_request:approve	✗	✗	✗	✓	✓

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
job_request:reject	✗	✗	✗	✓	✓
job_request:assign-engineer	✗	✗	✗	✓	✓

3.5 Job Cards

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
job_card:read-list	✗	✓	✓	✓	✓
job_card:read-detail	✗	✓	✓	✓	✓
job_card:start-work	✗	✗	✓ (own)	✗	✓
job_card:update-tasks	✗	✗	✓ (own)	✗	✓
job_card:complete	✗	✗	✓ (own)	✗	✓
job_card:verify-close	✗	✗	✗	✓	✓
job_card:reopen	✗	✗	✗	✓	✓
job_card:generate-pdf	✗	✓	✓	✓	✓

3.6 Dashboard & Inquiry

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
dashboard:view	✓	✓	✓	✓	✓

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
inquiry:search-vendors	✓	✓	✓	✓	✓
inquiry:search-products	✓	✓	✓	✓	✓
inquiry:search-job-cards	✗	✓	✓	✓	✓
inquiry:search-instruments	✓	✓	✓	✓	✓

3.7 Master Data (formerly Admin module — now Super Admin only)

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
master:employees:manage	✗	✗	✗	✗	✓
master:vendors:manage	✗	✗	✗	✗	✓
master:equipment-types:manage	✗	✗	✗	✗	✓
master:divisions:manage	✗	✗	✗	✗	✓
master:lookup-values:manage	✗	✗	✗	✗	✓

3.8 Audit & Logs

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
audit_log:read	✗	✗	✗	✗	✓

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
export:trigger (PDF, future Excel)	✗	✗	✓	✓	✓

Row-level visibility still applies: Normal User sees only **their own** job requests. View-Only sees all. Lab roles see all.

Section 4 — Updated Personas (5, not 6)

#	Persona Role	Daily Use	Removed/New?
1	Ravi Lab Engineer	Executes jobs (5-10×/day)	unchanged
2	Meena Lab In-charge	Triage + verify (all day)	unchanged
3	Karthik Normal User	Raise requests (1-2×/week)	unchanged
4		—	REMOVED
5	Suresh Super Admin	Role assignment + master data + system integrity (now broader)	role broadened
6	Anjali View-Only	Audit/management oversight	unchanged

Section 7 — Updated Module Responsibility Matrix

Module	Primary Actors	Notes
Dashboard	All roles	Role-filtered KPIs
Job Requests	Normal User → Lab In-charge	unchanged
Job Cards	Lab Engineer, Lab In-charge	unchanged
Equipment	Normal/Engineer/In-charge/Super Admin	create open to all (except View-Only); verify/condemn restricted
Schedule (Phase 2)	Lab In-charge	unchanged
Procurement (Phase 2)	Lab In-charge, Super Admin	(was: Admin → now Super Admin)
Inquiry	All roles	role-filtered results
Reports (Phase 2)	Lab In-charge, Super Admin	(was: Admin → now Super Admin)
Master Data Mgmt (Phase 2)	Super Admin only	absorbs former Admin module

Section 12 — Summary

Topic	Final State
Roles	5 — Super Admin, Lab In-charge, Lab Engineer, Normal User, View-Only

Topic	Final State
Personas	5 — Suresh, Meena, Ravi, Karthik, Anjali
Admin role	Removed — responsibilities absorbed by Super Admin
Equipment create permission	Super Admin + Lab In-charge + Lab Engineer + Normal User
Equipment verify permission	Lab In-charge + Super Admin
Master data CRUD	Super Admin only (Phase 2 UI; manual DB edits in MVP)
Bootstrap	Min 2 Super Admin employee IDs via env CSV
All prior docs	Reconciled — diagrams, matrix, rules, FRs, MVP split all consistent now

Your Move

Answer the **5 confirmation points in Section 11** (silence = approved) and we move to **Phase 3: Database Design** — the heaviest, most important phase.

For Phase 3 I'll need you to share the **existing 64-table schema** (a `mysqldump --no-data` or `SHOW CREATE TABLE` for each is ideal). Take your time gathering it — we can also start Phase 3 by drafting the **target schema** first and reconcile against existing tables once you share them.

Locked in. 5 roles. Clean model. Moving.  

finally what we
did till now
generate all
the things in
my response
style

7:05 PM

Claude responded: CMC MIS_SIMPLIFIED — Master Reference Document

Architected comprehensive master reference consolidating all project phases

Architected comprehensive master reference consolidating all project phases

CMC MIS_SIMPLIFIED — Master Reference Document

Consolidating Phases 0 → 2 · Single source of truth · Updated through May 15, 2026

This document supersedes every prior message. Anything not in here is not locked. Anything in here **is** locked unless we explicitly revise it.

Table of Contents

Part	Section	Topic
I — Foundation	1–4	Identity, Journey, Scope, Domain
II — People & Access	5–7	Roles, Permissions, Personas
III — Functional Design	8–12	Modules, Journeys, Rules, Requirements
IV — Technical Architecture	13–23	System, Network, Code Structure, Stack, Security
V — Domain Model	24–26	State Machines, Entities, Known Gaps
VI — Execution Plan	27–30	Risks, Roadmap, Decisions, Next
VII — Wrap-up	31	Summary Notes

PART I — FOUNDATION

❏ Project Identity (At a Glance)

Attribute	Value
Name	CMCMIS_SIMPLIFIED
Full form	Computerized Maintenance & Calibration Management Information System (Simplified)
Domain	Metrology · calibration · maintenance of T&ME and F&PE equipment
Environment	High-precision engineering (ISRO SAC-like)
Type	Internal web application
Deployment	On-prem / org private infrastructure
Owner	Deep Sorathiya (DS), Software Developer Intern
Team	Solo dev
Timeline	10 weeks
Architecture style	3-tier monolith (SPA + REST API + RDBMS)
Tech stack (BE)	Node.js + Express
Tech stack (FE)	React + Vite + TailwindCSS
Database	MySQL 8.x (phpMyAdmin tool)
Auth (v1)	Employee-ID + DB password
Auth (future)	Active Directory SSO-ready
Compliance posture	Architecture-aware (audit-ready), NABL/ISO not enforced v1

2 Phase Journey So Far

- ☒ PHASE 0 — Discovery & Requirements Capture [DONE]
 - ☒ PHASE 1 — Domain Modeling & Ideation [DONE]
 - ☒ PHASE 2 — System Architecture & Tech Stack [DONE]
 - ☐ PHASE 3 — Database Design (60+ tables) [NEXT]
 - ☐ PHASE 4 — API Contract Design
 - ☐ PHASE 5 — Module Implementation
 - ☐ PHASE 6 — Security Hardening + Testing
 - ☐ PHASE 7 — Scaling, Caching, Observability
 - ☐ PHASE 8 — Deployment
 - ☐ PHASE 9 — Handover Documentation
-

3 Scope — Locked

3.1 What's IN the 10-Week MVP

Scope	Items
MVP modules	Auth + RBAC · Equipment master & register & verify · Job Requests · Job Cards · Dashboard · Inquiry
Cross-cutting	PDF generation (job card + cal cert) · Audit logs (basic) · Responsive UI · Row-level visibility
Auth	Employee-ID + password against DB · JWT + refresh cookie · 5-role RBAC
Bootstrap	≥2 Super Admin employee IDs seeded via env

3.2 What's in Phase 2 (Post-Internship)

Schedule (PM + Cal calendar) · Procurement (POs + spares) · Reports (analytics + exports) · Master Data Management UI (Super Admin only) · Notifications (in-app feed)

3.3 What's in Backlog (Future)

SSO/AD integration · Email/SMTP notifications · Mobile/tablet optimized views · Barcode/QR · Backup & DR · NABL/ISO calibration certificate templates · Multi-language · Bulk import/export · External API

3.4 Architectural Simplifiers (your decisions that pay back)

Simplifier	Consequence
No file storage (PDFs generated on-demand)	No upload pipeline, no virus scanning, no storage quotas
No email/SMTP	No async queues, no notification service — all "notifications" are in-app
No Redis	Stick to optimized SQL + pagination + connection pooling
No mobile-first	Desktop-primary; responsive viewport is sufficient

4 Domain Knowledge Captured

4.1 T&ME vs F&PE

Term	Meaning	Examples	Calibration?
T&ME	Test & Measurement Equipment	Oscilloscopes, multimeters, spectrum analyzers, power meters	 Required

Term	Meaning	Examples	Calibration?
F&PE	Fabrication & Production Equipment	Soldering stations, vacuum chambers, environmental test chambers, ESD setups	Optional

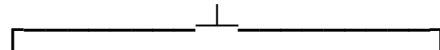
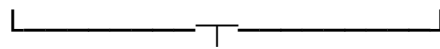
4.2 Core Metrology Concepts (Acknowledged, Lightly Enforced v1)

Concept	Meaning	v1 Treatment
Calibration	Compare instrument vs reference standard	Tracked via Job Cards
Traceability chain	Cal → standard → national reference (NIST/NABL)	Schema-ready, not enforced
Out-of-tolerance (OOT)	Reading outside acceptable limits	Captured as observation
Uncertainty	Statistical confidence on measurement	Captured as field, no formal budget calc
Calibration certificate	Legal output of every cal	Generated as PDF, downloadable
Environmental conditions	Temp/humidity at cal time	Captured fields in job card
Recall	Standards failure → re-evaluate dependent cals	Out of v1 scope

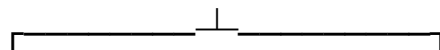
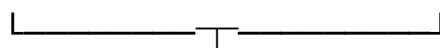
PART II — PEOPLE & ACCESS

5 Role Model — Final (5 Roles)

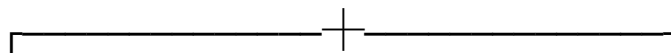
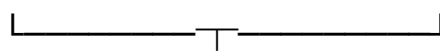
| SUPER ADMIN | ← role assignment + master data +
system config



| LAB IN-CHARGE | ← approves, assigns, verifies,
schedules



| LAB ENGINEER | ← executes jobs, fills cards



| NORMAL USER | | VIEW-ONLY USER |

| (raises req.) | | (read-only) |



Note: "Admin" role removed; responsibilities absorbed into Super Admin.

6 Permission Matrix — Complete & Final

6.1 Auth & Identity

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
auth:login / logout / refresh	✓	✓	✓	✓	✓
me:read (own profile)	✓	✓	✓	✓	✓

6.2 User & Role Management

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
user:read-list	✗	✗	✗	✗	✓
user:role-assign	✗	✗	✗	✗	✓
user:activate / deactivate	✗	✗	✗	✗	✓

6.3 Equipment

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
equipment:read-list / detail	✓	✓	✓	✓	✓
equipment:create	✓	✗	✓	✓	✓
equipment:update	✗	✗	✓	✓	✓
equipment:verify (PENDING→ACTIVE)	✗	✗	✗	✓	✓
equipment:condemn	✗	✗	✗	✓	✓
equipment:delete (hard)	✗	✗	✗	✗	✓

6.4 Job Requests

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
job_request:create	✓	✗	✓	✓	✓
job_request:read-own	✓	✓	✓	✓	✓
job_request:read-all	✗	✓	✓	✓	✓
job_request:approve / reject	✗	✗	✗	✓	✓
job_request:assign-engineer	✗	✗	✗	✓	✓

6.5 Job Cards

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
job_card:read-list / detail	✗	✓	✓	✓	✓
job_card:start / update-tasks / complete (own only)	✗	✗	✓	✗	✓
job_card:verify-close	✗	✗	✗	✓	✓
job_card:reopen	✗	✗	✗	✓	✓
job_card:generate-pdf	✗	✓	✓	✓	✓

6.6 Dashboard, Inquiry, Master Data, Audit

Resource:Action	Normal	View-Only	Lab Engineer	Lab In-charge	Super Admin
dashboard:view	✓	✓	✓	✓	✓
inquiry:search-*	✓	✓	✓	✓	✓
master*:manage	✗	✗	✗	✗	✓
audit_log:read	✗	✗	✗	✗	✓
export:trigger	✗	✗	✓	✓	✓

Row-level rule: Normal User sees only **own** job requests. All others see all.

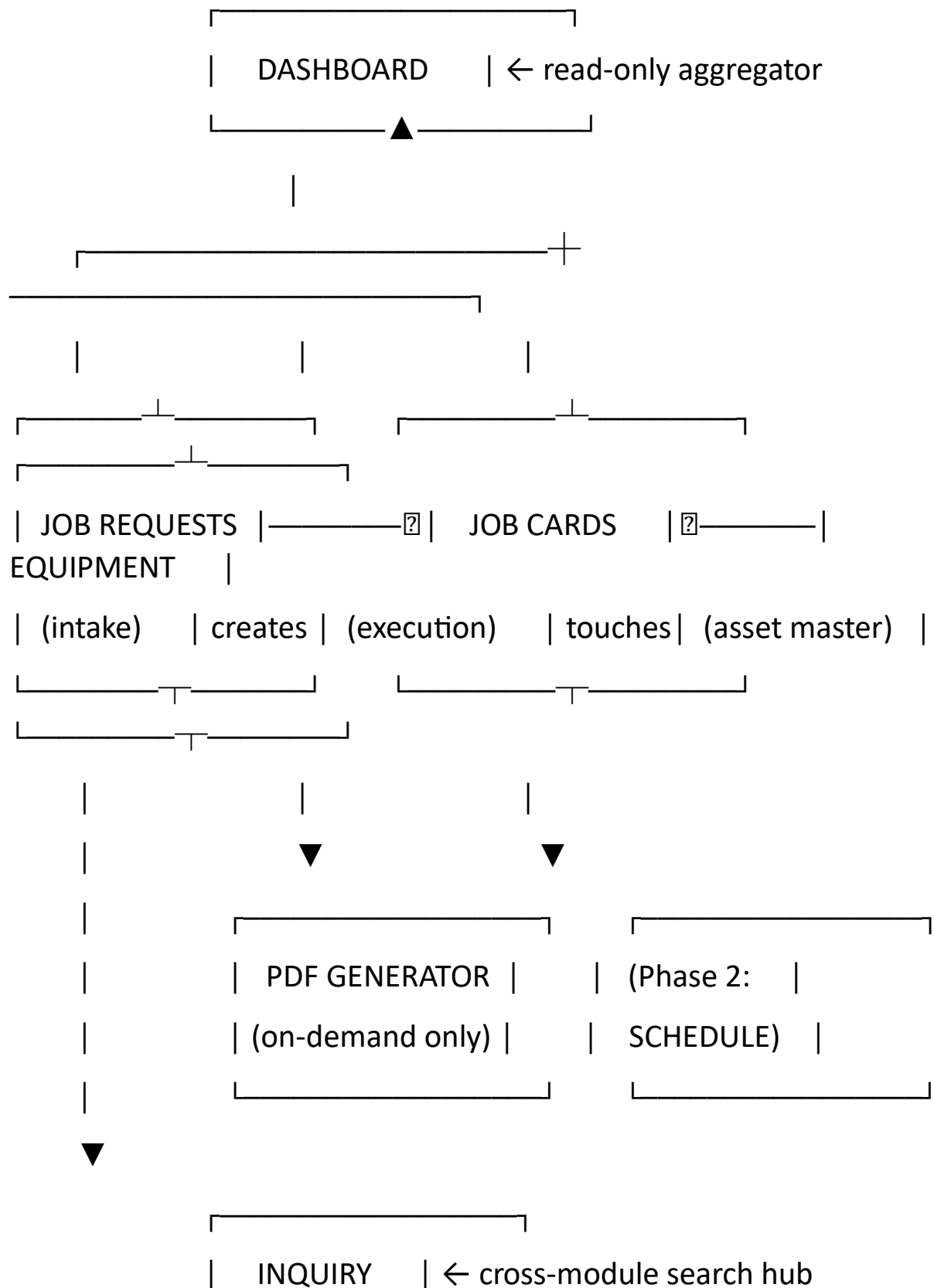
🗒️ Personas — Final 5

#	Persona	Role	Frequency	Primary Goal
1	Suresh	Super Admin	Weekly	Role assignment, master data, system integrity
2	Meena	Lab In-charge	All day	Triage, assign, verify, schedule
3	Ravi	Lab Engineer	5–10× per day	Execute jobs, fill cards
4	Karthik	Normal User	1–2× per week	Raise & track requests
5	Anjali	View-Only User	Weekly/monthly	Audit, oversight

PART III — FUNCTIONAL DESIGN

8 Module Map & Responsibility Matrix

8.1 Module Interconnection Diagram

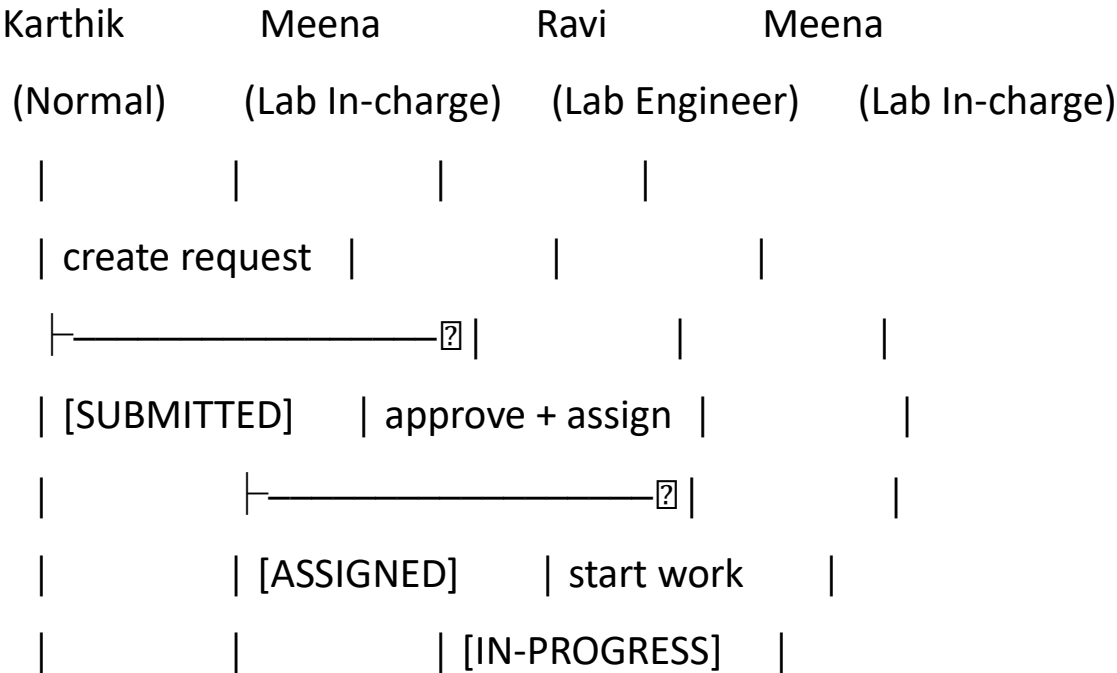


8.2 Module Ownership (MVP only)

Module	Owns Data	Primary Actors
Dashboard	nothing (read-only)	all roles
Job Requests	job_requests	Normal → Lab In-charge
Job Cards	job_cards, tasks, observations	Lab Engineer, Lab In-charge
Equipment	equipment + specs + cal history	4 roles create; In-charge/SA verify
Inquiry	nothing (read-only)	all roles
Auth/RBAC	users, roles, permissions	Super Admin

9 Three Critical User Journeys (MVP Demo)

9.1 Journey A — Request to Close



- | └— calibration due alerts (30 days + overdue)
 - | └— workload chart (In-charge)
 - | └— recently updated jobs
 - |
 - └─ ? Inquiry (4 tabs)
 - └— Vendor lookup
 - └— Product/equipment lookup
 - └— Job card status
 - └— Instrument by serial
-

10 Business Rules Catalogue (Consolidated)

10.1 Auth & RBAC

ID	Rule
BR-AUTH-01	Login by employee_id only (no email/username)
BR-AUTH-02	User must exist in employee directory — no self-registration
BR-AUTH-03	No role → defaults to Normal User
BR-AUTH-04	JWT 15 min; refresh token 7 days
BR-AUTH-05	≥2 Super Admin employee IDs seeded via SUPER_ADMIN_EMPLOYEE_IDS env CSV

ID	Rule
BR-AUTH-06	All login attempts logged
BR-AUTH-07	User can be deactivated (status=INACTIVE), preserving history
BR-RBAC-01	Only Super Admin assigns roles
BR-RBAC-02	Single primary role per user (v1)
BR-RBAC-03	Permissions = User → Role → Permission, never by role name in code
BR-RBAC-04	Sidebar + routes filtered by permissions
BR-RBAC-05	Every API endpoint enforces permission check
BR-RBAC-06	View-Only cannot trigger writes
BR-RBAC-07	Role changes take effect on next login or refresh

10.2 Equipment

ID	Rule
BR-EQP-01	Serial number unique system-wide
BR-EQP-02	Equipment ∈ {T&ME, F&PE}

ID	Rule
BR-EQP-03	F&PE: calibration optional
BR-EQP-04	T&ME: calibration frequency required
BR-EQP-05	$\text{next_cal_due_date} = \text{last_cal_date} + \text{frequency}$
BR-EQP-06	Status transitions follow equipment state machine
BR-EQP-07	Hard delete = Super Admin only; condemn = In-charge or Super Admin
BR-EQP-08	Search = case-insensitive on serial/model/manufacturer/type
BR-EQP-09	Every registration: registered_by, registered_at, verified_by (nullable), verified_at
BR-EQP-10	New equipment defaults to PENDING_VERIFICATION; verified by In-charge/Super Admin

10.3 Job Requests

ID	Rule
BR-JR-01	Must reference an existing equipment
BR-JR-02	$\text{Type} \in \{\text{Calibration, Repair, Registration}\}$
BR-JR-03	DRAFT possible before submission

ID	Rule
BR-JR-04	After SUBMIT, only Lab In-charge moves state
BR-JR-05	Must be assigned to a Lab Engineer to become Job Card
BR-JR-06	"Submitted by" auto-filled from current user (non-overridable)
BR-JR-07	High-priority repair surfaces at top of queue
BR-JR-08	Rejection requires reason (text + reason code)

10.4 Job Cards

ID	Rule
BR-JC-01	Job card auto-created when request approved + assigned
BR-JC-02	Lifecycle: ASSIGNED → IN-PROGRESS → COMPLETED → VERIFIED/CLOSED
BR-JC-03	Only assigned engineer moves to IN-PROGRESS or COMPLETED
BR-JC-04	Only Lab In-charge verifies/closes
BR-JC-05	Closed → REOPENED by In-charge with reason
BR-JC-06	Tasks configurable per job type

ID	Rule
----	------

BR-JC-07	Cal jobs need before/after readings + env conditions before COMPLETE
----------	--

BR-JC-08	State history is append-only
----------	------------------------------

10.5 PDF / Audit / Security / Visibility

ID	Rule
----	------

BR-PDF-01	PDFs generated from DB on demand; nothing stored
-----------	--

BR-PDF-02	Job card PDF: header, equipment, tasks, observations, signatures (textual), date
-----------	--

BR-PDF-03	Cal cert PDF: equipment, standards, readings, env, uncertainty placeholders, valid-until
-----------	--

BR-PDF-04	All PDFs: generation timestamp + "Generated by CMC MIS" + record ID
-----------	---

BR-AUD-01	Every write op on critical tables → audit_logs
-----------	--

BR-AUD-02	Audit log captures: who, what, when, before, after, IP, UA
-----------	--

BR-AUD-03	Exports logged with user + record IDs
-----------	---------------------------------------

BR-AUD-04	Sensitive fields filtered out per permission
-----------	--

BR-AUD-05	HTTPS in prod; JWT in header, refresh in httpOnly cookie
-----------	--

BR-VIS-01	Normal User sees only own job requests
-----------	--

BR-VIS-02	Lab Engineer sees own assignments + queue
-----------	---

BR-VIS-03	Lab In-charge + Super Admin see all
-----------	-------------------------------------

ID	Rule
BR-VIS-04	View-Only sees all (no writes)
BR-MASTER-01	All master data CRUD = Super Admin only

11 Functional Requirements (MVP)

Module	Count	Examples
Auth & RBAC (FR-A)	10	Login, JWT issue, sidebar filter, role assignment, /me endpoint
Equipment (FR-E)	9	List, search, register, edit, verify, color-coded due dates, history
Job Requests (FR-JR)	8	List, filter, create, draft, approve, reject, auto-create card on approve
Job Cards (FR-JC)	8	List, detail with stepper, tasks, observations, transitions, PDF, timeline
Dashboard (FR-D)	6	KPIs, cal due alerts, workload, status pie, recent jobs, quick actions
Inquiry (FR-I)	4	4-tab search, debounce, drill-down, empty states
TOTAL	45 FRs	All MUST priority for v1

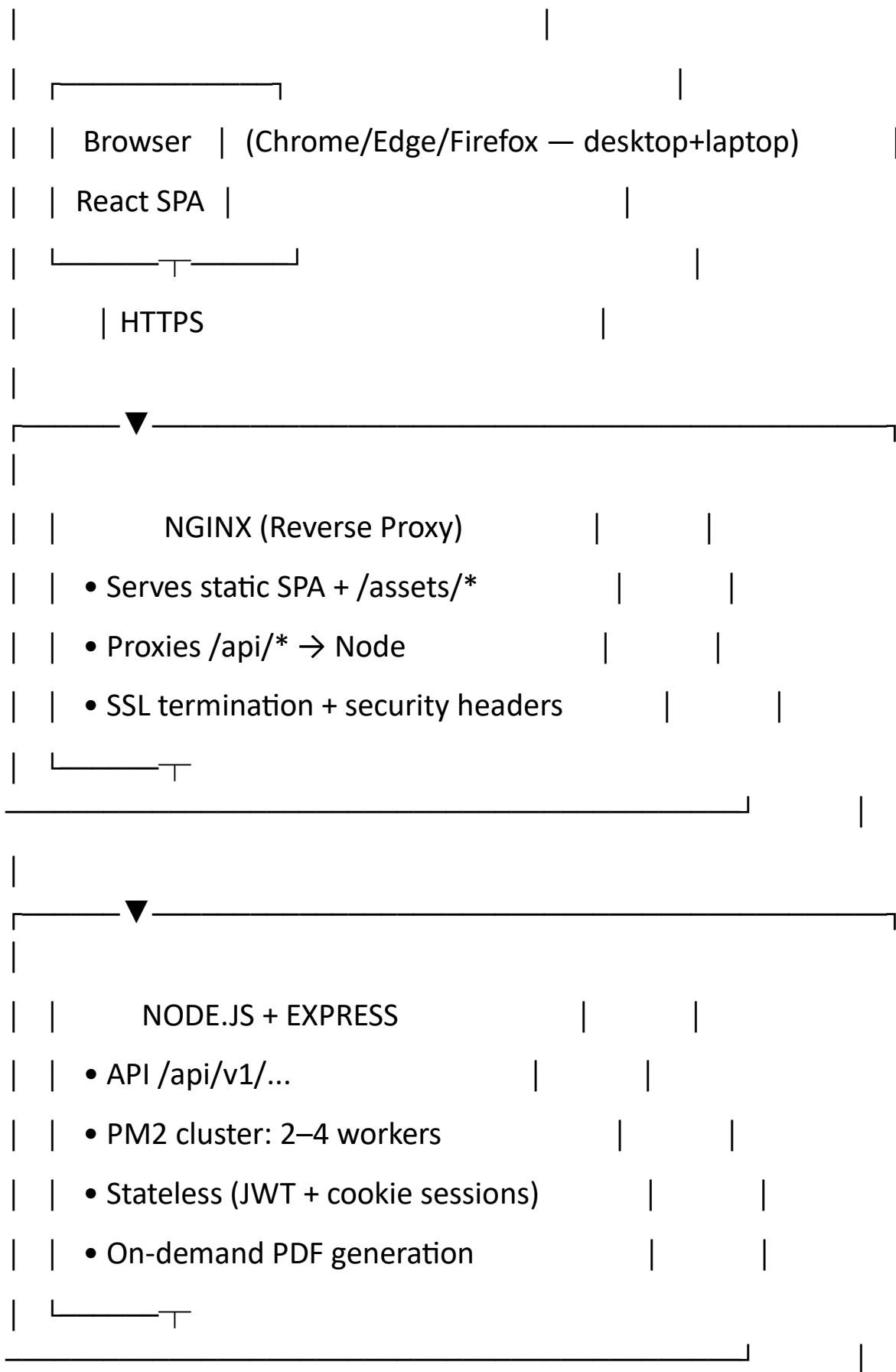
12 Non-Functional Requirements

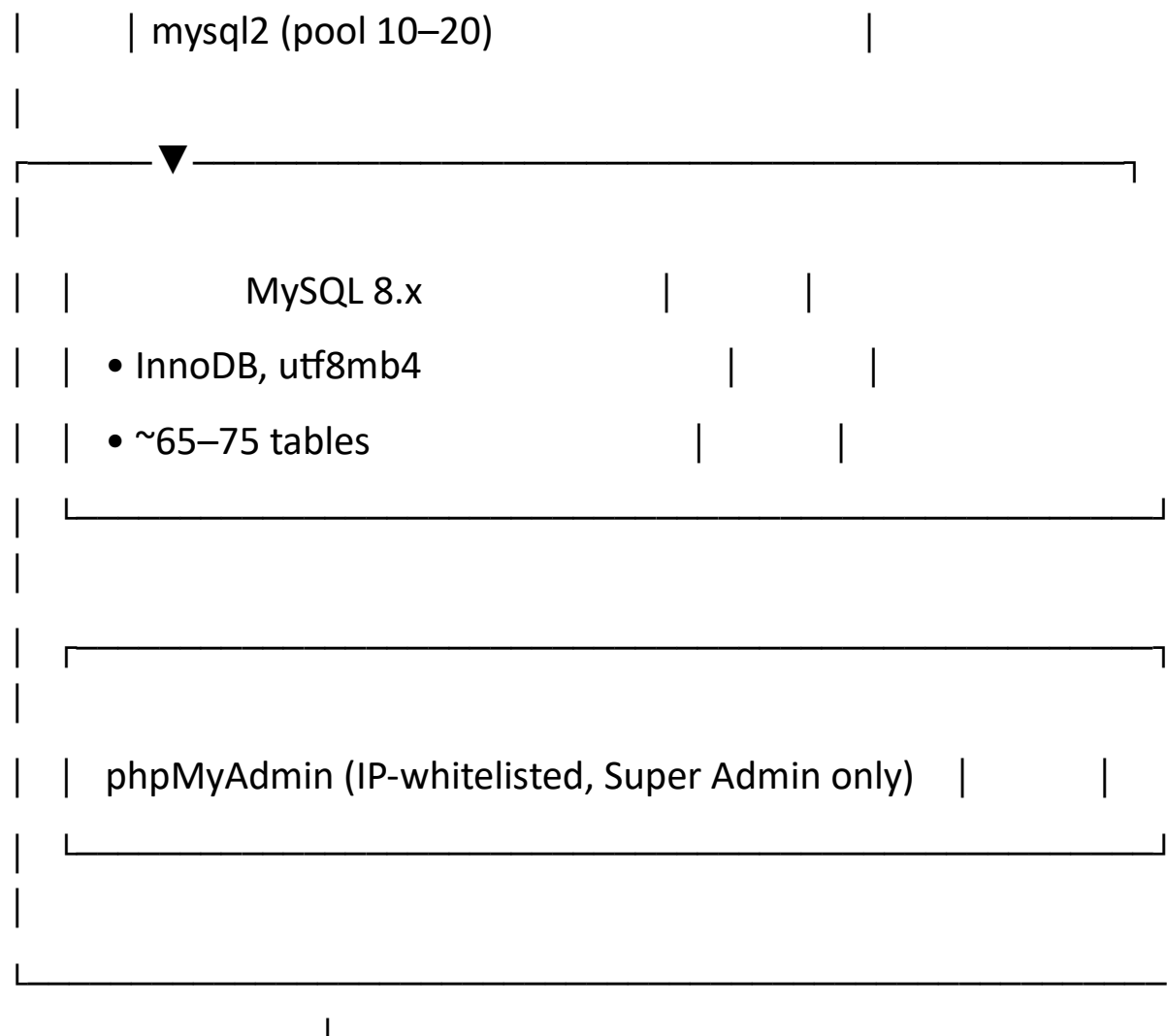
Category	Target
Perf — cold load	< 3 sec on intranet
Perf — API p95	< 500 ms
Pagination	25 default, 100 max
Sec — passwords	bcrypt, ≥10 rounds
Sec — JWT	15 min lifetime
Sec — refresh	7 days, httpOnly cookie, SameSite=Lax
Sec — SQLi	parameterized queries only
Sec — XSS	React auto-escape + CSP
Reliability	99% uptime business hours (post go-live)
DB pool	10–20 conns
Usability	≤3 clicks to any feature
Viewport	1280–1920 primary; degrades to 768
Audit	Every state-changing op logged
Browsers	Chrome/Edge/Firefox latest 2 versions
API versioning	/api/v1/... from day 1

PART IV — TECHNICAL ARCHITECTURE

13 System Architecture Diagram







14 Network & Deployment Topology

Layer	Process	Internal Port	Externally Reachable?
Nginx	systemd	443 (+ 80 redirect)	☒ to LAN/VPN
Node	PM2, 2–4 workers	3000 (loopback)	☐
MySQL	systemd	3306 (loopback)	☐

Layer	Process	Internal Port	Externally Reachable?
phpMyAdmin	Nginx vhost	8080 (IP whitelist)	⚠ admin only

Environments: development (laptop) → staging (org test) → production (live).

15 Folder Structure — Backend & Frontend

15.1 Backend Skeleton

backend/src/

- ├— config/ (env, db, logger, constants)
- ├— db/ (migrations, seeders, migrate.js)
- ├— routes/v1/ (auth, users, equipment, jobRequests, jobCards, dashboard, inquiry)
- ├— controllers/ (THIN — HTTP only)
- ├— services/ (THICK — business logic)
- ├— repositories/ (SQL only)
- ├— middleware/ (auth, permissions, validate, error, logger, rate-limit)
- ├— validators/ (zod schemas)
- ├— errors/ (ApiError + subclasses)
- ├— pdf/ (pdfkit templates)
- ├— utils/ (date, pagination, hash, response)
- └— app.js

Layered flow: **Route → Controller → Service → Repository → MySQL**

15.2 Frontend Skeleton (Feature-Based)

frontend/src/

- ├── api/ (axios client + per-resource api files)
- ├── components/
- | └── ui/ (Button, Input, Modal, Table, Pagination, Badge, Spinner)
- | └── layout/ (AppShell, Sidebar, Header, Breadcrumbs)
- | └── shared/ (StatusBadge, KpiCard, DataTable, PermissionGate)
- ├── features/
- | └── auth/
- | └── equipment/
- | └── jobRequests/
- | └── jobCards/
- | └── dashboard/
- | └── inquiry/
- ├── hooks/ (useAuth, usePermissions, useDebounce)
- ├── store/ (zustand stores)
- ├── routes/ (AppRouter, ProtectedRoute, PermissionRoute)
- ├── utils/, constants/, styles/
- └── App.jsx / main.jsx

Each feature folder = pages/ + components/ + hooks/ + (optional) store/.

16 API Design Conventions

Concern	Convention
---------	------------

Base URL	/api/v1/...
----------	-------------

List	GET /api/v1/equipment?page=1&pageSize=25&q=...&type=...
------	--

Create	POST /api/v1/equipment
--------	------------------------

Detail	GET /api/v1/equipment/:id
--------	---------------------------

Update	PUT /api/v1/equipment/:id
--------	---------------------------

State transition	POST /api/v1/job-cards/:id/transition
------------------	---------------------------------------

PDF	GET /api/v1/job-cards/:id/pdf
-----	-------------------------------

Success envelope	{ success: true, data, pagination?, meta }
------------------	--

Error envelope	{ success: false, error: { code, message, details }, meta }
----------------	---

Status codes	200/201/204 · 400/401/403/404/409 · 500
--------------	---

Auth header	Authorization: Bearer <jwt>
-------------	-----------------------------

Refresh	httpOnly cookie
---------	-----------------

17 Final Tech Stack

17.1 Backend Libraries

Concern	Library
---------	---------

Framework	express ^4
-----------	------------

DB driver	mysql2/promise ^3
-----------	-------------------

Validation	zod ^3
------------	--------

Auth	jsonwebtoken ^9 + bcryptjs ^2
------	-------------------------------

Logger	pino ^8 (+ pino-pretty dev)
--------	-----------------------------

Env	dotenv ^16 + envalid ^8
-----	-------------------------

Date	dayjs ^1
------	----------

PDF	pdfkit ^0.14
-----	--------------

Rate limit	express-rate-limit ^7
------------	-----------------------

Security	helmet ^7 + cors ^2
----------	---------------------

Process	pm2
---------	-----

Quality	eslint + prettier + husky + lint-staged
---------	---

17.2 Frontend Libraries

Concern	Library
---------	---------

Build	vite
-------	------

Framework	react 18
-----------	----------

Routing	react-router-dom v6
---------	---------------------

Server state	@tanstack/react-query
--------------	-----------------------

Client state	zustand
--------------	---------

HTTP	axios
------	-------

Concern	Library
Forms	react-hook-form + zod
UI	custom + tailwindcss v3
Icons	lucide-react
Charts	recharts
Date	dayjs
Toasts	sonner

17.3 Deliberately NOT Picked

Redux · Sequelize/Prisma/TypeORM · GraphQL · TypeScript (v1) · Material-UI/Ant Design · Next.js · Mongoose

~~18~~ Critical Decision — Raw SQL over ORM

Chosen: Raw SQL (mysql2/promise) + Repository Pattern + manual SQL migrations.

Why	Detail
Existing 64-table DB	ORMs work best when they own the schema
Metrology complexity	Window funcs, CTEs, conditional aggregates — fight an ORM
Solo dev	Fewer abstractions = less mental juggling
Learning value	SQL muscle compounds
Debugging	You see the actual query

Trade-off: more repository boilerplate → mitigated by shared helpers (db.query, db.queryOne, db.transaction).

19 Security Architecture (Layered)

Browser

|



HTTPS / TLS (Nginx)

HSTS, CSP, XFO (helmet)

CORS allowlist

|



Rate limit (express-rate-limit; tight on /login)

|



JWT verification (auth middleware)

Permission check (permissions middleware)

|



Zod input validation (body/query/params)

|



Business logic + audit logging

|



Parameterized SQL (mysql2)

|



MySQL

OWASP-T10 coverage: SQLi (params), AuthN (JWT+bcrypt+rate-limit), AuthZ (perm middleware), XSS (React+CSP), Misconfig (helmet+env), Logging (audit), Deserialization (zod).

20 PDF Strategy

Aspect	Choice
--------	--------

Library	pdfkit (over Puppeteer)
---------	--------------------------------

Pattern	Stream from service → response (never buffer full PDF)
---------	--

Filename	<doc-type>-<id>-<YYYYMMDD>.pdf
----------	--------------------------------

Audit	Every export logged in export_audit
-------	-------------------------------------

Storage	None — all PDFs derived from DB
---------	--

21 Logging & Error Handling

Concern	Approach
---------	----------

Logger	Pino, structured JSON
--------	-----------------------

Levels	error, warn, info, debug
--------	--------------------------

Per-request log	method, URL, status, duration, requestId, userId
-----------------	--

Forbidden in logs passwords, tokens, full bodies on auth routes

Request ID	propagated through logs + response envelope
------------	---

Error class hierarchy:

ApiError

- └─ ValidationError (400)
- └─ UnauthorizedError (401)
- └─ ForbiddenError (403)
- └─ NotFoundError (404)
- └─ ConflictError (409)

Central error middleware converts → standard error envelope;
unknown errors logged with stack, returned as generic 500.

~~22~~ Environment Config (Boot-Validated)

.env keys: NODE_ENV, PORT, DB_*, JWT_SECRET, JWT_EXPIRES_IN, REFRESH_SECRET, REFRESH_EXPIRES_IN, BCrypt_ROUNDS, CORS_ORIGINS, SUPER_ADMIN_EMPLOYEE_IDS (CSV ≥2), LOG_LEVEL, rate-limit knobs.

Validation = invalid → app crashes at boot if any required key missing.

~~23~~ Request Lifecycle (One Mental Model for Everything)

Browser → Nginx → Express

- requestLogger
- helmet + cors + body-parse
- rate-limit
- Route match
 - authMiddleware (verify JWT, attach req.user)
 - permissionsMiddleware (check resource:action)

- validateMiddleware (zod parse body/query/params)
- Controller (thin: extract, call service, respond)
- Service (business logic, state machine, transactions)
 - Audit log write
 - Repository (parameterized SQL)
 - MySQL
- Response envelope → Browser

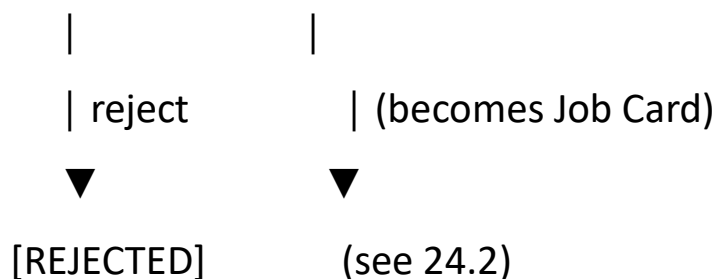
Memorize this. Every bug lives somewhere in this pipeline.

PART V — DOMAIN MODEL

~~24~~ State Machines

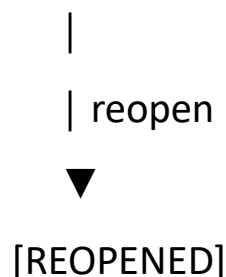
24.1 Job Request

[DRAFT] —submit—> [SUBMITTED] —assign—> [ASSIGNED]

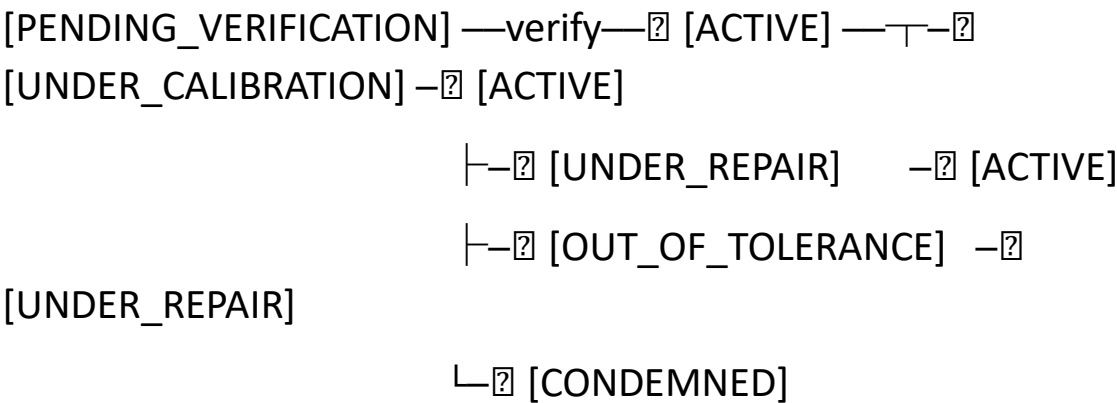


24.2 Job Card

[ASSIGNED] —start—> [IN-PROGRESS] —complete—> [COMPLETED]
 —verify—> [VERIFIED/CLOSED]



24.3 Equipment



24.4 Calibration Status (display badges)

Status	Visual
VALID	Green
DUE_SOON (<30 days)	Yellow
OVERDUE	Red
OUT_OF_TOLERANCE	Black
NOT_REQUIRED	Grey

25 Entity Universe (Precursor to Schema)

#	Cluster	Approx. Tables	Examples
1	Identity & Access	8–10	users, roles, permissions, role_permissions, user_roles, sessions, login_audit
2	Organization	4–5	divisions, departments, employees, designations, locations

#	Cluster	Approx. Tables	Examples
3	Equipment Master	8–10	equipment, types, categories, specs, status, manufacturers, models, docs, history
4	Job Lifecycle	6–8	job_requests, types, job_cards, tasks, observations, readings, attachments, state_history
5	Calibration	6–8	cal_records, cal_standards, traceability, cal_certs, env_conditions, uncertainty, OOT, recall
6	Maintenance	4–5	maint_records, types, PM plans, breakdowns, AMC
7	Scheduling	3–4	schedules, events, recurrence, notifications
8	Procurement	6–7	vendors, vendor_categories, POs, PO_lines, spares, stock, movements
9	Documents	3–4	docs, versions, types, signatures
10	Audit & Logs	3–4	audit_logs, change_history, login_audit, export_audit
11	Notifications	3	notifications, prefs, log
12	Lookups	5–7	units, statuses, frequencies, system_types
13	Reporting	2–3	saved_reports, runs, scheduled
	TOTAL	~65–75 tables	—

26 Critical Gaps in Original Spec (Resolved/Logged)

#	Gap	Status
1	Cal certificates	☑ In scope (PDF on demand)
2	Traceability chain	☐ Schema-ready, not enforced v1
3	OOT handling	☐ Captured as observation v1
4	Uncertainty	☐ Field present, no formal budget
5	Audit logs	☑ In scope (basic v1)
6	Approval workflow	☑ Defined in state machine
7	Notification spec	☐ In-app only v1
8	Doc storage	☑ No storage — generated on demand
9	Retention	☐ Indefinite until org defines
10	SSO protocol	☐ Plug-in later (AD-ready)
11	Backup / DR	🌀 Open — must discuss Phase 8
12	Concurrency / scale	☐ ~50–200 concurrent users assumed
13	Print cal cert	☑ In scope (PDF)
14	Barcode/QR	✗ Out of scope
15	Mobile/tablet	✗ Out (responsive only)
16	Classified data	☐ Row-level + secure sessions + limited exports

#	Gap	Status
17	NABL/ISO 17025	☐ Architecture-aware only

PART VI — EXECUTION PLAN

~~21~~ Risk Register (Live)

#	Risk	Status	Mitigation
R1	Scope creep	 Active	MVP discipline; backlog anything new
R2	64 existing tables unknown	☐ Active	Phase 3 starts with schema audit
R3	No backup infra	☐ Logged	Discuss Phase 8
R4	No QA support (solo)	☐ Active	Weeks 8–9 reserved for QA + bug bash
R5	Stakeholder feedback delayed	☐ Active	Demo every 2 weeks if possible
R6	Solo dev burnout	☐ Active	Sleep > rushed code; weekly milestones
R7	Compliance reqs surface late	☐ Active	Architecture stays compliance-friendly
R8	Raw SQL slows initial dev	☐ Active	Repository helpers
R9	Audit log latency	☐ Low	Async write

#	Risk	Status	Mitigation
R10	Single Super Admin = bottleneck	☐ Active	≥2 seeded

~~28~~ 10-Week Roadmap

WEEK PHASE MUST BE DONE BY END OF WEEK

-
- 1 Foundation Repo + skeleton runs end-to-end (React ↔ Express ↔ MySQL)
 - 2 DB Design Final schema deployed; seed data; migration script works
 - 3 Auth + RBAC 3 test users (different roles) log in, see different sidebars
 - 4 Equipment Register, list, search, view detail; verify flow works
 - 5 Job Requests DRAFT → SUBMITTED → APPROVED auto-creates Job Card
 - 6 Job Cards Full lifecycle + PDF generation works end-to-end
 - 7 Dash + Inquiry KPIs render live data; 4-tab inquiry returns real results
 - 8 Polish App "feels finished" — no broken empty/error states
 - 9 Hardening Audit logs everywhere; security review; query indexes
 - 10 Demo Prep Seeded demo data; docs; runbook; clean stakeholder demo

Discipline rule: if it isn't on the line for the current week, it goes to backlog. No exceptions.

29 Master Decision Ledger

#	Decision	Locked
D1	JavaScript + JSDoc + Zod (no TypeScript v1)	✓
D2	Raw SQL + Repository Pattern (no ORM)	✓
D3	React Query for server state	✓
D4	Zustand for client state	✓
D5	pdfkit for PDFs	✓
D6	Pino for logging	✓
D7	Routes → Controllers → Services → Repositories	✓
D8	Feature-based FE folders	✓
D9	Nginx reverse proxy in production	✓
D10	New equipment defaults to PENDING_VERIFICATION	✓
D11	≥2 Super Admin employee IDs seeded	✓
D12	5 roles (Admin removed, absorbed by Super Admin)	✓
D13	Equipment create open to 4 roles (not View-Only)	✓
D14	Equipment verify = Lab In-charge or Super Admin	✓
D15	Master data CRUD = Super Admin only	✓
D16	Lookup values seeded in Week 2; UI in Phase 2	✓

#	Decision	Locked
	D17 JWT 15 min + refresh cookie 7 days, httpOnly	✓
	D18 Bootstrap via .env SUPER_ADMIN_EMPLOYEE_IDS CSV	✓
	D19 All PDFs streamed (never buffered fully)	✓
	D20 Audit log on every state-changing op	✓

~~30~~ What's Next — Phase 3

Phase 3: Database Design. This is the heaviest, most decision-dense phase.

30.1 Phase 3 Deliverables

1. Existing schema audit (you share dump; we triage keep/modify/drop)
2. Cluster-by-cluster ER design (13 clusters from Section 25)
3. CREATE TABLE scripts (PKs, FKs, indexes, constraints, audit cols)
4. Seed data scripts (roles, permissions, super admins, lookups)
5. Migration runner (versioned SQL files)
6. ER diagram (visual)

30.2 To Begin Phase 3, You Will Share

- Existing 64-table DB structure (mysqldump --no-data or SHOW CREATE TABLE outputs)
- Short note on: which tables are **definitely in use**, **probably dead**, or **unsure**
- 2+ employee IDs you'll seed as Super Admin

If sharing the existing DB takes time, we can **start Phase 3 from a clean target design** and reconcile once you bring the existing schema. Either order works.

PART VII — WRAP-UP

Summary Notes (Pocket Card)

Category	Snapshot
System	CMCMIS_SIMPLIFIED · monolith · on-prem · MySQL · Node+Express · React+Vite+Tailwind
Timeline	10 weeks · solo dev · industry-grade target
Roles	5 — Super Admin, Lab In-charge, Lab Engineer, Normal User, View-Only
MVP modules	Auth/RBAC · Equipment · Job Requests · Job Cards · Dashboard · Inquiry
Out of MVP	Schedule · Procurement · Reports · Master Data UI · Notifications
Auth	Employee-ID + DB · JWT 15m + refresh 7d cookie · SSO-ready
DB access	Raw SQL + Repository Pattern · mysql2/promise
PDFs	pdfkit · streamed · no storage
Permissions	3-layer RBAC (user→role→permission→resource:action) · always enforced
State machines	Job Request · Job Card · Equipment (3 explicit machines)

Category	Snapshot
Audit	Every write op logged (who/what/when/before/after/IP)
Phase done	0 ☒ · 1 ☒ · 2 ☒
Phase next	3 — Database Design (target schema + existing schema reconciliation)
Decisions locked	20
Open items	Backup/DR strategy · NABL details · existing schema audit
Biggest risk	Scope creep — enforced by weekly milestone discipline

Your Move

Two things to unblock Phase 3:

1. **Confirm:** any final disagreement on the 20 decisions in Section 29? (Silence = approved.)
2. **Choose path:**
 - **Path A:** You share existing DB dump first → I reconcile into final schema.
 - **Path B:** I draft the target schema (clean, ideal) → we reconcile with your existing tables after.

Tell me **A** or **B**, and we begin Phase 3. 

 **Note:** This document is long-form on purpose — it's your single-source-of-truth reference for the remaining 10 weeks. If you want it as a downloadable Markdown file you can keep open in a second monitor, say the word and I'll generate it.